

User prompt:

also the next thing i want to ask is it possible to use a classification model instead of a regression model

ChatGPT said:

Excellent question — yes, **you absolutely can use a classification model** instead of a regression model, **but only if your prediction target can be meaningfully expressed as discrete classes** rather than continuous toxin concentrations.

Let's go step-by-step, because this decision changes everything: preprocessing, loss, metrics, and evaluation logic.

1. Core idea

- Your current **regression model** predicts a **continuous variable**, e.g. `dsp_toxins` concentration (e.g., $42 \rightarrow 983$ $\mu\text{g/kg}$).
- A **classification model** would predict a **category** such as:
 - `low` vs `high`
 - `safe` vs `unsafe`
 - or even multiple levels (e.g. `low`, `medium`, `high`, `very high`).

You'd convert continuous values into **bins (classes)**, then train the model to predict which class the next week falls into.

2. When classification makes sense

Classification is reasonable if:

- You care about *risk levels* or *alerts* (e.g., shellfish closures above 160 $\mu\text{g/kg}$ regulatory threshold).
- The data has too much noise for precise numeric regression.
- You're interested in accuracy / sensitivity / recall, not in exact concentrations.

It's less ideal if:

- You need continuous forecasts for downstream modeling.
- You want to capture subtle concentration changes, not just threshold exceedances.



3. How to convert your regression target into classes

Option A — Binary classification

Label based on a regulatory or biological threshold:

```
threshold = 160 # µg/kg, example regulatory limit
df['toxin_class'] = (df['dsp_toxins'] > threshold).astype(int)
```

→ 0 = safe, 1 = toxic.

Option B — Multi-class classification

Discretize concentration into multiple bins:

```
bins = [0, 100, 160, 300, 600, np.inf]
labels = ['very_low', 'low', 'moderate', 'high', 'very_high']
df['toxin_class'] = pd.cut(df['dsp_toxins'], bins=bins,
labels=labels)
```

You can choose bins based on percentiles or domain thresholds.



4. Model architecture and training changes

Your current LSTM architecture works fine — you only need to adjust the **output layer and loss**:

Task	Output layer	Activation	Loss function	Metrics
Regression	Dense(1)	None	MSE / MAE / Huber	MAE, RMSE, R ²

Binary classification	Dense(1)	Sigmoid	Binary crossentropy	Accuracy, F1, ROC-AUC
Multi-class classification	Dense(n_classes)	Softmax	Categorical crossentropy	Accuracy, F1, confusion matrix

Example for binary classification:

```
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
```

Multi-class:

```
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

5. Label alignment and preprocessing

Your existing windowing logic still works the same — you just change what *y* represents.

Currently:

```
y_target = target_col[t+time_steps : t+time_steps+horizon]
```

You'd instead extract the **class label** at that horizon:

```
y_target = toxin_class[t+time_steps + (horizon - 1)]
```

or one-hot encode for multi-class:

```
from tensorflow.keras.utils import to_categorical
y = to_categorical(y, num_classes=n_classes)
```

6. Evaluation metrics for classification

Instead of MAE/RMSE:

- **Accuracy** (overall correctness)
- **Precision / Recall / F1-score**
- **ROC-AUC**
- **Confusion matrix**

These help measure false alarms vs. missed detections — important for ecological forecasting (e.g., false safe prediction vs. false alert).

7. Advantages vs. disadvantages



Pros

Easier to interpret (e.g., “toxic week: yes/no”)

More robust to noise

Easier to evaluate for management purposes

Simplifies decision-making (alerts, closures)



Cons

Loses information about magnitude

Threshold choice may bias model

Less suitable for research needing concentration values

You can't compute MAE or RMSE anymore

8. Hybrid option: Regression + threshold classification

You can **keep regression outputs** and then **apply a threshold to predicted values** for classification metrics.

That lets you:

- Train on continuous toxin values.
- Post-process predictions into classes.
- Evaluate both MAE and confusion matrix.

This is often done in environmental forecasting because it combines fine-grained prediction with threshold-based alerts.

Example:

```
y_pred_class = (y_pred_inv > 160).astype(int)
y_true_class = (y_true_inv > 160).astype(int)
confusion = confusion_matrix(y_true_class, y_pred_class)
```

✓ 9. What this means for your project

You have three realistic paths forward:

Goal	Best Model Type	Notes
Predict exact DSP toxin values	Regression (what you're doing now)	Use MAE/RMSE metrics
Predict exceedance of threshold (e.g., >160 µg/kg)	Binary classification	Use sigmoid + crossentropy
Predict risk levels (low/med/high)	Multi-class classification	Use softmax + categorical crossentropy
Combine both (predict value, then classify)	Regression + post-thresholding	Best of both worlds

Would you like me to:

1. Show you exactly how to **modify your existing LSTM code** into a binary classifier version, or
2. Show how to **derive a classification layer from your regression model's outputs** (hybrid thresholding approach)?

User Prompt:

in this case we would be predicting if that toxin level is above or below that regulatory limit which is 160

ChatGPT said:

Perfect — that's exactly the **binary classification** setup ✓

So your goal becomes:

Given the last N weeks of features, predict whether next week's DSP toxin level will be **above (unsafe)** or **below (safe)** the regulatory threshold of 160 $\mu\text{g/kg}$.

Let's break down **exactly how this works** step by step and then I'll show you what to change in your current pipeline.

⚙️ Step 1. Define the classification target

Instead of predicting continuous toxin values (`dsp_toxins`), you'll create a **binary label** column.

```
threshold = 160 # regulatory limit
df["toxin_class"] = (df["dsp_toxins"] > threshold).astype(int)
```

- 0 → safe (≤ 160)
- 1 → unsafe (> 160)

This will now become your **new target** variable.

✂️ Step 2. Keep your windowing logic the same

Your existing sliding-window preprocessing stays exactly the same — it will still generate samples like:

```
X: last 12 weeks of features
y: class label for next 4 weeks (horizon)
```

Except now, instead of having numeric values in `y`, you'll have binary 0/1 values.

You might choose to predict **only $t+1$** (the next week's class) at first, since multi-step classification ($t+1$, $t+2$, $t+3$, $t+4$) is trickier.

Example:

```
y_all.append(group["toxin_class"].iloc[i + time_steps]) #
binary label
```

Step 3. Modify the model output

Your LSTM layers can remain identical — you just change the **final dense layer and loss function**:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

model = Sequential([
    LSTM(64, input_shape=(time_steps, num_features),
        return_sequences=False),
    Dense(1, activation='sigmoid') # binary classifier
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', 'Precision', 'Recall']
)
```

- The `sigmoid` activation outputs a probability between 0–1.
- The model learns to minimize cross-entropy loss (how far off its predicted probabilities are).
- Metrics like accuracy, precision, and recall are much more meaningful here than MAE/RMSE.

Step 4. Make predictions

After training, your model outputs probabilities:

```
y_pred_prob = model.predict(X_test)
```

Convert to binary classes:

```
y_pred_class = (y_pred_prob > 0.5).astype(int)
```

Step 5. Evaluate performance

Now you can build a **confusion matrix** and compute **classification metrics**:

```
from sklearn.metrics import confusion_matrix,
classification_report, roc_auc_score

cm = confusion_matrix(y_true, y_pred_class)
print("Confusion matrix:\n", cm)
print(classification_report(y_true, y_pred_class))
print("ROC-AUC:", roc_auc_score(y_true, y_pred_prob))
```

These tell you:

- How many times the model predicted *toxic* correctly (**True Positives**)
- How many times it missed a toxic week (**False Negatives**)
- How often it triggered a false alarm (**False Positives**)

Step 6. Optional — balance the classes

DSP toxin data is usually **imbalanced** (most weeks are safe).
To handle that:

Use **class weights** during training:

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

classes = np.unique(y_train)
class_weights = compute_class_weight('balanced',
classes=classes, y=y_train)
class_weights = dict(zip(classes, class_weights))

model.fit(X_train, y_train, epochs=..., batch_size=...,
class_weight=class_weights)
```


-
- Or oversample the minority class (e.g. SMOTE).

This ensures the model doesn't just learn to predict "safe" 100% of the time.

✅ Step 7. Interpretation and application

The final output can be interpreted as **"probability of a toxic event next week."**

You can threshold it differently depending on your tolerance for risk:

- 0.5 → equal weighting
 - 0.3 → more sensitive (fewer missed blooms, more false alarms)
 - 0.7 → more conservative (fewer false alarms, more missed blooms)
-

📌 Summary

Component	Regression	Classification (yours)
Target	Continuous toxin value	0 = safe, 1 = toxic
Output	Real number	Probability (0–1)
Activation	None / Linear	Sigmoid
Loss	MAE, MSE, Huber	Binary crossentropy
Metrics	MAE, RMSE, R ²	Accuracy, F1, ROC-AUC
Output interpretation	Predicted concentration	Toxic risk probability

Would you like me to generate a **ready-to-run version** of your existing `run_model_single_site.py` script modified to perform this binary classification (predicting if > 160)?

User Prompt:

well now that it is the case of classification does the windowing still matter i do not know

ChatGPT said:

Thought for 4 seconds

Short answer: **Yes — windowing still matters for classification.**

Windowing is *how* you give the model temporal context (what happened in the last N weeks) so it can predict whether the *next* week (or weeks) will be above the threshold. Everything I said about the window size tradeoffs for regression still applies: context vs. sample count, overlap, seasonality, missing data, etc.

Below I'll explain why windowing still matters for classification, what changes (and what doesn't), practical choices, and paste in concrete code you can drop into your pipeline to produce binary labels and train/evaluate a binary LSTM classifier.

Why windowing still matters (conceptual)

1. **Temporal context = predictive signal.**

A single row (this week) often doesn't contain the precursors of an exceedance. The last N weeks (trend, rise, persistence) help the model detect patterns that precede exceedances.

2. **Overlapping windows increase effective training examples.**

Sliding windows create many supervised samples from one long time series (e.g., weeks 1–12 → label t+1, then weeks 2–13 → label t+2). That lets the model learn from many “moments”.

3. **Window length controls bias/variance tradeoff.**

- Too short → model misses slow-onset signals.
- Too long → fewer samples, risk of overfitting and more missing-values within windows.

4. **Class imbalance & windowing interaction.**

If exceedances are rare, some windows will have label=1 rarely. Window size affects how many positive windows you get (e.g., longer windows might still only have rare positives).

5. Horizon choice matters.

Predicting $t+1$ (one-week ahead) usually needs shorter windows than predicting $t+4$; but in practice you test several window/horizon combos.

What changes vs. what stays the same

- **Stays same:** sliding-window creation, per-site grouping (to avoid leakage), normalization decisions, chronological splitting.
 - **Changes:** y becomes binary (0/1) instead of continuous. Model output/loss/metrics change (sigmoid + binary_crossentropy, accuracy/precision/recall/F1/AUC). Evaluation uses confusion matrices and ROC, not MAE/RMSE.
-

Concrete recommendations

1. Start by predicting **$t+1$** (one-week ahead) — simpler, usually more reliable. Later you can expand to multi-horizon (multi-output).
 2. Keep the same sliding-window approach; label each window by whether the target week (first of the horizon, or last depending on definition) exceeds threshold.
 3. Choose window sizes to balance context and sample count — try candidates (4, 8, 12, 24) and pick by validation F1 / ROC-AUC (not accuracy if classes are imbalanced).
 4. Use **class weights** or resampling (oversample minority) during training to handle imbalance.
 5. Use probability thresholds tuned on validation (you don't have to use 0.5).
-

How to modify your pipeline — minimal edits

1) Create binary labels in preprocessing

Add this right after you have `dsp_toxins` cleaned:

```
THRESHOLD = 160 # µg/kg

# after load_and_merge_sites / clipping etc.
df_all["toxin_class"] = (df_all["dsp_toxins"] >
THRESHOLD).astype(int)
# keep -1 handling as before; if dsp_toxins == -1 you might set
toxin_class = -1 or np.nan
# but better: only create windows where target is valid (your
filter_valid_samples will remove -1 targets)
```

2) Change `make_supervised` to return binary labels (for $t+1$)

If you want to predict $t+1$ class (one output), change `y_all.append(...)` to append the class of the first horizon step:

```
# inside make_supervised
# currently: y_all.append(data[i + time_steps:i + time_steps +
horizon, target_idx])
# replace with:
class_idx = group.columns.get_loc("toxin_class")
# then in loop:
y_all.append(data[i + time_steps + (0), class_idx]) # t+1 class
(0 or 1)
# if you want to predict multi-horizon binary labels (t+1..t+4),
you can create a vector:
# y_all.append(data[i + time_steps:i + time_steps + horizon,
class_idx]) # shape (horizon,)
```

If you're using the previously provided `make_supervised` that expects `target="dsp_toxins"`, add an explicit option. Example small patch:

```
def make_supervised(df, features, target, time_steps=12,
horizon=4, class_target=None, predict_horizon_index=0):
    ...
    # if class_target is not None, use class_target column for
y; else use numeric target per original
    if class_target is not None:
        class_idx = group.columns.get_loc(class_target)
```

```

        y_all.append(data[i + time_steps +
predict_horizon_index, class_idx])
    else:
        y_all.append(data[i + time_steps:i + time_steps +
horizon, target_idx])

```

3) Model output and training (binary)

Example using your existing LSTM builder but with binary output:

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dropout, Dense
from tensorflow.keras.optimizers import Adam

def build_binary_lstm(input_shape, params):
    model = Sequential([
        LSTM(params["units"], input_shape=input_shape),
        Dropout(params["dropout"]),
        Dense(params["dense"], activation="relu"),
        Dense(1, activation="sigmoid")    # binary
    ])
    model.compile(loss="binary_crossentropy",
                  optimizer=Adam(learning_rate=params["lr"]),
                  metrics=["accuracy"])
    return model

```

4) Class weights (handle imbalance)

Compute class weights from `y_train`:

```

from sklearn.utils.class_weight import compute_class_weight
classes = np.unique(y_train)    # should be [0,1]
cw = compute_class_weight("balanced", classes=classes,
y=y_train)
class_weight = dict(zip(classes, cw))
# then during fit:
model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=params['epochs'], batch_size=params['batch'],
          class_weight=class_weight, callbacks=callbacks)

```

5) Predictions and evaluation

```
y_pred_prob = model.predict(X_test).ravel() # probabilities
# choose threshold (default 0.5 or tune using validation ROC)
threshold = 0.5
y_pred_class = (y_pred_prob >= threshold).astype(int)

from sklearn.metrics import classification_report,
confusion_matrix, roc_auc_score
print(classification_report(y_test, y_pred_class))
print("ROC-AUC:", roc_auc_score(y_test, y_pred_prob))
cm = confusion_matrix(y_test, y_pred_class)
```

If your `y_test` shape is (samples, horizon) because you made multi-horizon binary labels, select the horizon you evaluate (e.g., `y_test[:,0]` for t+1).

Multi-horizon classification options

1. **Single-horizon models:** Train separate classifiers for t+1, t+2, etc. (simplest).
2. **Multi-output model:** Final Dense with `horizon` units, `sigmoid` activation, and `binary_crossentropy` (use `from_logits=False`).
 - Loss needs masking if some horizons are invalid (-1). You can use sample weighting or a custom masked loss.
3. **Sequence-to-sequence classifier:** LSTM returning sequences + TimeDistributed Dense — more complex.

For your current pipeline, I recommend start with t+1 binary classifier.

Practical example: small end-to-end snippet

This assumes you already loaded `X_train`, `y_train`, ... where `y_*` are binary labels for t+1.

```
# build
```

```

params = {"units": 64, "dropout": 0.2, "dense": 32, "lr": 1e-3,
"epochs": 100, "batch": 32}
input_shape = (X_train.shape[1], X_train.shape[2])
model = build_binary_lstm(input_shape, params)

# class weights
from sklearn.utils.class_weight import compute_class_weight
cw = compute_class_weight("balanced",
classes=np.unique(y_train), y=y_train)
class_weight = dict(zip(np.unique(y_train), cw))

# fit
history = model.fit(X_train, y_train, validation_data=(X_val,
y_val),
                    epochs=params["epochs"],
batch_size=params["batch"],
                    class_weight=class_weight, verbose=1)

# predict & eval
y_prob = model.predict(X_test).ravel()
from sklearn.metrics import classification_report,
confusion_matrix, roc_auc_score
y_pred = (y_prob >= 0.5).astype(int)
print(classification_report(y_test, y_pred))
print("ROC AUC:", roc_auc_score(y_test, y_prob))
print("Confusion:\n", confusion_matrix(y_test, y_pred))

```

How to pick the window now (classification-specific notes)

- **Same process** as regression: ACF/PACF, sample-count calculation, grid search by validation metric (prefer F1, ROC-AUC, recall depending on what you care about).
- If missing positives are rare, prefer a slightly larger window only if it meaningfully improves recall — otherwise smaller windows to increase sample count.
- Tune threshold on validation to optimize recall vs precision depending on operational priorities (missed toxic event is worse than a false alarm? choose threshold for high recall).

Final practical tips

- Start with **t+1**, **time_steps=12**, train the binary model, check **precision/recall/F1/ROC**.
- If you get very low recall (miss many toxic events), increase window, add features (seasonal sin/cos, lag features), or use class weights/oversampling.
- If you get many false positives, tune threshold upward and inspect feature importance/SHAP (optional).
- If you want, I can:
 - Patch your exact preprocessing + **make_supervised** to emit **toxin_class** and produce **X_train/y_train** for t+1; **or**
 - Generate a complete runnable **run_binary_lstm.py** that fits into your existing preprocessed directories and outputs confusion matrices/ROC plots.

Which would you like me to produce now?

User Prompt: [copy of preprocessing script]

so what would i need to change in the preprocessing

""" Paper-faithful preprocessing for ANN/LSTM models

```
----- Implements Section 2.2.2 exactly as in Cruz et al.
(2022): - Merge 8 sites: RIAV1, L5b, ETJ1, LAL, L7c1, LAG, POR2, FAR1 - Weekly
resampling - Handle missing values: * ≤25% missing → linear interpolation * >25%
missing → fill with -1 - Clip negative DSP toxins - Normalize all variables to [0,1] (global
across sites), leaving -1 untouched - Split by years: Train (first 2.5 yrs), Val (next 1.5
yrs), Test (last 1 yr) - Transform into supervised learning structure (per site!): X:
[samples, 12, features] y: [samples, 4, 1] - Generate datasets: univariate, bivariate,
multivariate """ import pandas as pd import numpy as np import glob import os import
json # ===== CONFIG ===== #
INPUT_PATH = "all_sites/*.xlsx" OUTPUT_DIR = "preprocessed"
os.makedirs(OUTPUT_DIR, exist_ok=True) SITES_USED = ["RIAV1", "L5b", "ETJ1",
"LAL", "L7c1", "LAG", "POR2", "FAR1"] EXPECTED_COLS = [ "dsp_toxins",
"dsp_phyto", "mean_sst", "mean_chl", "mean_temp", "max_temp", "min_temp",
"mean_wind_intensity", "wind_dir", "rainfall" ] TIME_STEPS = 12
FORECAST_HORIZON = 4 #
===== # def
```



```

load_and_merge_sites(input_path): all_dfs = [] for file in glob.glob(input_path):
site_code = os.path.basename(file).split("_")[0] if site_code not in SITES_USED:
continue df = pd.read_excel(file, parse_dates=["Date"]).set_index("Date").sort_index() df
= df.resample("W-WED").mean() # Ensure all expected cols exist for col in
EXPECTED_COLS: if col not in df.columns: df[col] = np.nan # Clip negative toxins
df["dsp_toxins"] = df["dsp_toxins"].clip(lower=0) # Handle missing values missing_frac =
df[EXPECTED_COLS].isna().mean() for col in EXPECTED_COLS: if missing_frac[col]
<= 0.25: df[col] = df[col].interpolate(method="time", limit_direction="both") else: df[col] =
df[col].fillna(-1) df["Site"] = site_code all_dfs.append(df) df_all =
pd.concat(all_dfs).sort_index() df_all[EXPECTED_COLS] =
df_all[EXPECTED_COLS].fillna(-1) return df_all def global_normalize(df, cols): scalers =
{} for col in cols: valid_vals = df.loc[df[col] != -1, col] if valid_vals.empty: col_min,
col_max = 0.0, 1.0 else: col_min, col_max = valid_vals.min(), valid_vals.max()
scalers[col] = {"min": float(col_min), "max": float(col_max)} if col_max > col_min: df[col]
= np.where( df[col] == -1, -1, (df[col] - col_min) / (col_max - col_min) ) else: df[col] = 0.0
return df, scalers def chronological_split(df): """Assign splits globally by date."""
start_date, end_date = df.index.min(), df.index.max() train_weeks = int(2.5 * 52)
test_weeks = 52 val_weeks = 293 - (train_weeks + test_weeks) train_end = start_date +
pd.Timedelta(weeks=train_weeks) val_end = train_end +
pd.Timedelta(weeks=val_weeks) def get_split(date): if date <= train_end: return "train"
elif date <= val_end: return "val" else: return "test" df["split"] =
df.index.to_series().apply(get_split) return df # no prediction occurs in this function # this
takes 12 weeks as the input for the window and then 4 as the output # the window then
slides by one week e.g. (input: weeks 1 - 12, output: 13-16 -> input: weeks 2-13,
output:14-17) # all values used in the window are the true observed values # windows
have a slight overlap as this helps generate context # windows are created site-by-site
to prevent cross-site leakage # the windows split the site into multiple samples def
make_supervised(df, features, target, time_steps=12, horizon=4): """Create supervised
samples per site to avoid cross-site leakage.""" X_all, y_all, dates_all, site_ids_all,
splits_all = [], [], [], [], [] for site, group in df.groupby("Site"): group = group.sort_index()
data = group[features + [target]].values target_idx = group.columns.get_loc(target)
dates = group.index.values split_col = group["split"].values for i in range(len(group) -
time_steps - horizon): X_all.append(data[i:i + time_steps, :len(features)])
y_all.append(data[i + time_steps:i + time_steps + horizon, target_idx])
dates_all.append(dates[i + time_steps]) site_ids_all.append(site)
splits_all.append(split_col[i + time_steps]) return ( np.array(X_all), np.array(y_all),
np.array(dates_all), np.array(site_ids_all), np.array(splits_all) ) def save_datasets(X, y,
dates, site_ids, splits, output_path): os.makedirs(output_path, exist_ok=True) for s in
["train", "val", "test"]: idx = splits == s np.save(os.path.join(output_path, f"X_{s}.npy"),
X[idx]) np.save(os.path.join(output_path, f"y_{s}.npy"), y[idx])
np.save(os.path.join(output_path, f"dates_{s}.npy"), dates[idx])
np.save(os.path.join(output_path, f"site_ids_{s}.npy"), site_ids[idx]) #
===== PIPELINE ===== # print("\nLoading
and merging all sites...") df_all = load_and_merge_sites(INPUT_PATH) print(f"Loaded
merged dataset with shape: {df_all.shape}") print("\nNormalizing globally...") df_all,
scalers = global_normalize(df_all, EXPECTED_COLS) print("\nApplying chronological

```

```
split...) df_all = chronological_split(df_all) # Save cleaned merged data
df_all.to_csv(os.path.join(OUTPUT_DIR, "merged_cleaned.csv")) with
open(os.path.join(OUTPUT_DIR, "scalers.json"), "w") as f: json.dump(scalers, f,
indent=4) print("\nCreating supervised datasets (12→4)...") VARIANTS = { "univariate":
["dsp_toxins"], "bivariate": ["dsp_toxins", "dsp_phyto"], "multivariate":
EXPECTED_COLS } for variant, feats in VARIANTS.items(): print(f" Processing
{variant}...") X, y, dates, site_ids, splits = make_supervised( df_all, feats,
target="dsp_toxins", time_steps=TIME_STEPS, horizon=FORECAST_HORIZON )
print(f" Total samples: {len(X)} ({np.sum(splits=='train')} train, " f"{np.sum(splits=='val')}
val, {np.sum(splits=='test')} test)") # DEBUG: print first 5 sequences print(f"\nSample of
first 5 y sequences for {variant}:") for i in range(min(5, len(y))): print(f"{site_ids[i]} |
{dates[i]} | y={y[i]}") save_datasets(X, y, dates, site_ids, splits,
os.path.join(OUTPUT_DIR, variant)) print("\n✅ Preprocessing complete.")
print("Datasets saved in:", OUTPUT_DIR) print("Variants: univariate, bivariate,
multivariate") print("Each contains: X_train, y_train, X_val, y_val, X_test, y_test (.numpy
files)")
```

ChatGPT said:

Excellent — this preprocessing script is already very clean and modular, so modifying it for **classification (above/below threshold)** is quite easy.

Let's go step-by-step so you understand exactly what changes and *why* each one is needed.

You'll only need to edit **two small sections** and optionally add one new parameter.

Goal

You want your model to predict whether **DSP toxins exceed 160 µg/kg**, not the numeric value.

So instead of:

```
y = [12, 15, 9, 22]    # continuous toxin values
```

you'll have:

```
y = [0, 0, 0, 1]       # binary labels (below/above threshold)
```

But you **still** use windowing to give the model context (past 12 weeks → predict if next week is above 160).

The key is changing how **y** is constructed — everything else (normalization, splitting, etc.) can stay.

✓ Changes to make

1 Add a binary “class” column after normalization and before splitting

Add this block right after:

```
print("\nNormalizing globally...")
df_all, scalars = global_normalize(df_all, EXPECTED_COLS)
```

Insert:

```
print("\nCreating binary toxin class column...")
THRESHOLD = 160
# Important: create class column *before* windowing
# We'll mark rows with missing (-1) toxins as NaN so they're
# ignored later
df_all["toxin_class"] = np.where(
    df_all["dsp_toxins"] == -1,
    np.nan,
    (df_all["dsp_toxins"] > THRESHOLD).astype(int)
)
```

That gives you a binary target column:

```
0 = below threshold, 1 = exceedance.
```

2 Modify `make_supervised()` to use binary labels for classification

Change this function to optionally accept a `class_target` argument (so you can reuse it later).

Here's a **drop-in replacement** for your `make_supervised()`:

```
def make_supervised(df, features, target, time_steps=12,
                    horizon=4, class_target=None):
    """
    Create supervised samples per site.
    If class_target is provided, y will be binary (0/1) using
    that column instead of numeric target.
    """
```

```

X_all, y_all, dates_all, site_ids_all, splits_all = [], [],
[], [], []

for site, group in df.groupby("Site"):
    group = group.sort_index()
    data = group[features + [target]].values
    dates = group.index.values
    split_col = group["split"].values

    # If using classification, pre-fetch binary labels
separately
    if class_target is not None:
        class_values = group[class_target].values

    for i in range(len(group) - time_steps - horizon):
        X_all.append(data[i:i + time_steps, :len(features)])

        if class_target is not None:
            # Predict if t+1 (first horizon step) exceeds
threshold
            y_all.append(class_values[i + time_steps])
        else:
            # Original regression target (4-step horizon)
            target_idx = group.columns.get_loc(target)
            y_all.append(data[i + time_steps:i + time_steps
+ horizon, target_idx])

        dates_all.append(dates[i + time_steps])
        site_ids_all.append(site)
        splits_all.append(split_col[i + time_steps])

    return (
        np.array(X_all),
        np.array(y_all),
        np.array(dates_all),
        np.array(site_ids_all),
        np.array(splits_all)
    )

```

3 Adjust the call inside your preprocessing loop

When you're creating each variant, add a **second loop** (or swap depending on what you want).

Example:

```
for variant, feats in VARIANTS.items():
    print(f"  Processing {variant} (classification)...")
    X, y, dates, site_ids, splits = make_supervised(
        df_all, feats, target="dsp_toxins",
        time_steps=TIME_STEPS, horizon=FORECAST_HORIZON,
        class_target="toxin_class"  # <-- NEW
    )

    print(f"  Total samples: {len(X)} ({np.sum(splits=='train')}
train, "
        f"{np.sum(splits=='val')} val,
{np.sum(splits=='test')} test)")

    # Cast y to int (0 or 1)
    y = y.astype(int)

    save_datasets(X, y, dates, site_ids, splits,
                  os.path.join(OUTPUT_DIR, variant +
"_classification"))
```

This will generate classification-ready datasets in:

```
preprocessed/univariate_classification/
preprocessed/bivariate_classification/
preprocessed/multivariate_classification/
```

Each containing:

```
X_train.npy, y_train.npy, X_val.npy, y_val.npy, X_test.npy,
y_test.npy
```

4 (Optional but recommended) filter invalid rows before windowing

To make sure missing (-1) toxin values don't create invalid labels:

Add right after you create the "toxin_class" column:

```
# Remove any rows where class label is NaN (due to missing toxin values)
df_all = df_all.dropna(subset=["toxin_class"])
```

✓ Summary of what changed

Section	Change	Purpose
After normalization	Add <code>toxin_class</code> binary column	Creates classification target
<code>make_supervised()</code>	Add optional <code>class_target</code> arg	Allows binary or regression
Loop	Pass <code>class_target="toxin_class"</code>	Build classification dataset
Optional	Drop NaNs	Remove invalid labels

🧠 Quick sanity check

If your test dataset originally had 66 regression samples, you'll now have **roughly the same number** for classification — unless you dropped NaN labels.

Then your LSTM classifier (as we discussed earlier) will take:

```
input_shape = (12, n_features)
output = Dense(1, activation='sigmoid')
loss = 'binary_crossentropy'
```

If you want, I can give you the **full edited preprocessing script** ready to copy-paste (with both regression and classification branches included).

Would you like me to generate that version?

User Prompt:

no i need you to explain in a very detailed manner what changes need to be made and why

ChatGPT said:

Thought for 10s

Great — I'll walk through **exactly** what to change in your preprocessing + why, with concrete options, code snippets, and the reasoning behind every decision. I'll cover:

1. what the label should be (options)
2. where to create it and why (timing)
3. how to change `make_supervised()` to produce classification y's (single-step vs multi-horizon)
4. how to handle missing values (-1), masks, and dropping samples
5. how this affects saved files / shapes / later model code
6. evaluation notes (class imbalance, metrics, confusion matrices)

I'll be explicit about every line you should add/replace and *why* — no vague talk.

1) Decide the label you want (important starting choice)

Two common options — pick one (I'll explain consequences for preprocessing & model):

A. Single-horizon exceedance (most common):

`y = 1` if `t+1` (the first forecast week after the 12-week input) `> 160`, else `0`.

- Pros: simple classification, model predicts next-week exceedance.
- Cons: ignores whether `t+2..t+4` exceed the limit.

B. Multi-horizon exceedance (multilabel):

`y = [b1, b2, b3, b4]` where `b1` indicates `t+1 > 160`, `b2` indicates `t+2 > 160`, etc.

- Pros: gives full horizon information.

- Cons: becomes multi-output binary (you'll need a sigmoid for each horizon and binary crossentropy per output).

Recommendation: start with **A (single-horizon)** — simpler, easier to diagnose. If you need to predict all 4 horizon exceedances later, you can extend to B.

I'll show code for **A** (single $t+1$) and note what to change for B.

2) *Where* to create the binary target and *why* (timing)

Always create the binary label AFTER normalization but BEFORE windowing.

Why:

- The label depends on `dsp_toxins` observed value (actual $\mu\text{g/kg}$). You want the threshold comparison to be on the real units (not normalized). In your current pipeline you normalized to $[0,1]$ **globally**. If you compare to 160 after normalization you must invert it — easier: create the class before normalizing or create it after normalization by comparing to an inverse-scaled value.
- To keep it simple and robust: create the class **before** normalization (so you use raw toxin $\mu\text{g/kg}$), or create it **after** normalization but compare against the inverse-scaled threshold. Both are fine; I'll recommend making it **after clipping negatives and handling missing values but before `global_normalize()`** (so label uses original units). That avoids subtle scaling logic.

Concrete placement in your script (after you load and resample each site's df, after clipping negatives and missing value handling but before concatenation/global normalize):

Right after you prepare `df` for a site in `load_and_merge_sites()` (after you set `df["Site"]`), add:

```
# create binary exceedance column in original units
THRESHOLD = 160.0
# Mark missing toxin rows (-1 was used later; at this point NaNs
may be present before filling)
# -- but ensure you compare against actual toxin values, not
normalized.
```



```
df['toxin_class'] = np.where(df['dsp_toxins'].isna(), np.nan,
(df['dsp_toxins'] > THRESHOLD).astype(int))
```

Or if you prefer to create after the full concatenation **and** after missing value handling/clip (which your code already does), create it after `df_all = pd.concat(all_dfs)` and **before** `global_normalize(...)`:

```
# After df_all = pd.concat(all_dfs).sort_index()
df_all['toxin_class'] = np.where(df_all['dsp_toxins'] == -1,
np.nan, (df_all['dsp_toxins'] > 160.0).astype(int))
```

Why mark NaN when toxin == -1?

Your pipeline uses `-1` as placeholder for missing after interpolation. Those rows should not produce labels (we don't know ground truth), so make class `np.nan` for those rows.

3) Modify `make_supervised()` to produce classification labels

You need `make_supervised()` to optionally return binary labels (single step or multi-step). Replace the current version with the following **drop-in** (I'll explain each branch):

```
def make_supervised(df, features, target, time_steps=12,
horizon=4, class_target=None, class_mode='single'):
    """
    Create supervised samples per site.
    - If class_target is None -> original regression behavior (y
    = horizon numeric values).
    - If class_target provided and class_mode=='single' -> y is
    binary label for t+1 (shape (N,) or (N,1))
    - If class_target provided and class_mode=='multi' -> y is
    binary vector length=horizon (shape (N, horizon))
    Note: we leave -1 placeholders in features as-is; samples
    with NaN labels must be dropped later.
    """
    X_all, y_all, dates_all, site_ids_all, splits_all = [], [],
    [], [], []

    for site, group in df.groupby("Site"):
```

```

        group = group.sort_index()
        # ensure column exists
        if class_target is not None and class_target not in
group.columns:
            raise KeyError(f"class_target '{class_target}' not
in dataframe columns for site {site}")

        data = group[features + [target]].values
        dates = group.index.values
        split_col = group["split"].values

        if class_target is not None:
            class_vals = group[class_target].values # may
contain np.nan for missing labels

        for i in range(len(group) - time_steps - horizon):
            X_all.append(data[i:i + time_steps, :len(features)])

            if class_target is None:
                # regression: collect horizon numeric targets
(unchanged)
                target_idx = group.columns.get_loc(target)
                y_all.append(data[i + time_steps:i + time_steps
+ horizon, target_idx])
            else:
                if class_mode == 'single':
                    # single-horizon binary label for t+1
                    y_all.append(class_vals[i + time_steps]) #
scalar (0/1) or nan
                elif class_mode == 'multi':
                    # multi-horizon: binary for each horizon
step
                    y_all.append(class_vals[i + time_steps:i +
time_steps + horizon])
                else:
                    raise ValueError("class_mode must be
'single' or 'multi'")

            dates_all.append(dates[i + time_steps])
            site_ids_all.append(site)

```

```

        splits_all.append(split_col[i + time_steps])

    # convert arrays
    X_arr = np.array(X_all)                # shape (N,
time_steps, n_features)
    y_arr = np.array(y_all)                # shape
depends on mode
    dates_arr = np.array(dates_all)
    site_ids_arr = np.array(site_ids_all)
    splits_arr = np.array(splits_all)

    return X_arr, y_arr, dates_arr, site_ids_arr, splits_arr

```

Why this structure?

- Reuses the same sliding-window logic (still site-by-site, avoiding cross-site leakage).
- If `class_target` provided, `y` is produced from the already-created column `toxin_class` (which contains 0/1 or NaN for missing). This preserves correctness (class uses real-world threshold).
- `class_mode='single'` is the simplest: predict t+1 exceedance. If you later choose multi-horizon, you can set `class_mode='multi'`.

4) How to handle windows whose label is missing or invalid

After building `X`, `y`, `dates`, `site_ids`, `splits`, you must **remove samples where the label is NaN** (i.e., where `toxin_class` was missing). For regression you already remove windows with missing targets using `filter_valid_samples()` — do a similar step for classification.

Add a helper function:

```

def filter_valid_class_samples(X, y, dates):
    """
    Remove samples where the class label is nan.
    - If y is shape (N,) or (N,1) -> treat nan entries

```

```

- If y is shape (N, horizon) -> treat any nan in the row as
invalid (unless you want to allow partial)
"""
y_arr = np.array(y)
if y_arr.ndim == 1:
    mask = ~np.isnan(y_arr)
else:
    # for single-horizon stored as (N,1) => squeeze
    if y_arr.shape[1] == 1:
        mask = ~np.isnan(y_arr.squeeze())
    else:
        # for multi-horizon: require ALL horizon labels
        present (or change to allow any)
        mask = ~np.any(np.isnan(y_arr), axis=1)

return X[mask], y_arr[mask], dates[mask]

```

Use this after you call `make_supervised()` for classification datasets — before saving arrays.

Why drop NaN-label windows?

You cannot train on a sample where the true label is unknown. Dropping them keeps training truthful and prevents confusing the model.

5) Save classification datasets in their own output locations (avoid clobber)

As you already intended to keep regression datasets, save classification datasets to a separate folder (e.g. `preprocessed/<variant>_classification/`) so you don't overwrite the regression ones.

Example usage in the main loop:

```

# classification branch example for univariate:
Xc, yc, dates_c, site_ids_c, splits_c = make_supervised(df_all,
feats, target='dsp_toxins',

time_steps=TIME_STEPS, horizon=FORECAST_HORIZON,

class_target='toxin_class', class_mode='single')

```

```
# filter invalid class samples
Xc, yc, dates_c = filter_valid_class_samples(Xc, yc, dates_c)
yc = yc.astype(int) # ensure integer labels

save_datasets(Xc, yc, dates_c, site_ids_c, splits_c,
os.path.join(OUTPUT_DIR, variant + "_classification"))
```

Note on y shape: for single-horizon `yc` will be 1D (N,) or 2D (N,1). For training with Keras, convert to shape `(N,1)` if using a single sigmoid output. For multi-output, keep `(N, horizon)`.

6) Model-side changes (summary of what to update once preprocessing is done)

When you switch to classification you must adjust training code:

For single-horizon (label shape `(N,1)`):

- Model final layer: `Dense(1, activation='sigmoid')`
- Loss: `binary_crossentropy`
- Metrics: `accuracy`, `AUC()`, `Precision`, `Recall` etc.
- `y` must be shape `(N,1)` and dtype `float32` or `int`.

Example model builder change:

```
from tensorflow.keras.losses import BinaryCrossentropy

model = Sequential([
    LSTM(units, input_shape=input_shape),
    Dropout(dropout),
    Dense(dense_size, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer=Adam(lr), loss=BinaryCrossentropy(),
metrics=['accuracy', tf.keras.metrics.AUC()])
```

For multi-horizon (label shape **(N, H)**):

- Final layer: `Dense(H, activation='sigmoid')`
- Loss: `binary_crossentropy` (Keras will compute per-output BCE and average)
- Metrics: per-output AUC/accuracy or overall average.

Additional important points:

- **Class imbalance:** exceedance events may be rare. Use `class_weight` in `model.fit()` or oversample minority, or use focal loss. For multilabel, you can compute per-output weights. Example simple `class_weight`:

```
from sklearn.utils.class_weight import compute_class_weight
cw = compute_class_weight('balanced',
classes=np.unique(y_train_flat), y=y_train_flat.ravel())
class_weight = {0: cw[0], 1: cw[1]}
model.fit(X_train, y_train, class_weight=class_weight, ...)
```

- **Evaluation:** use confusion matrices, precision/recall, F1, ROC-AUC. Save confusion matrices per split.
- **Masking:** you already have `-1` for missing features; these remain unchanged — classification labels are produced separately and you dropped NaN labels.

7) Concrete checklist & patch plan (summary you can follow)

1. **Add THRESHOLD variable** near top of preprocessing script (e.g. `THRESHOLD = 160.0`).
2. **Create `toxin_class` column** in `df_all` (after concatenation and missing handling, but before normalization OR do it earlier as explained). Code:

```
df_all['toxin_class'] = np.where(df_all['dsp_toxins'] == -1,
np.nan, (df_all['dsp_toxins'] > THRESHOLD).astype(int))
```

3. **(Optional)** Drop rows with NaN labels if you want fewer windows; or keep them and let `make_supervised()` drop windows whose label is NaN. I recommend the latter.
 4. **Replace `make_supervised()`** with the enhanced version I provided (supports `class_target` and `class_mode`).
 5. **Add `filter_valid_class_samples()`** helper and call it for classification outputs just after `make_supervised()`.
 6. **Save classification datasets** into separate subfolders (e.g. `preprocessed/univariate_classification`).
 7. **Update training script:**
 - final `Dense(1, activation='sigmoid')` for single-horizon
 - `loss=binary_crossentropy`
 - metrics include `AUC`, `Precision`, `Recall`
 - handle class imbalance with `class_weight` or oversampling.
 8. **Evaluation script:**
 - compute confusion matrices using `exceed_true` and `exceed_pred` — but now `exceed_pred` is the model's binary prediction (apply threshold 0.5 to sigmoid outputs)
 - ensure confusion matrices count equals number of samples used (you should see the count equal to number of valid y samples for that split).
-

8) Answers to likely follow-up questions you implicitly asked

- **Does windowing still matter for classification?**

Yes. The sliding windows still give the model temporal context (last 12 weeks) to decide whether next week ($t+1$) exceeds threshold — exactly the same reason it mattered for regression.

- **Do we still normalize features globally?**
Yes. Keep normalization for continuous features. The label is binary and **should not** be normalized.
 - **Will changing preprocessing change existing regression files?**
If you save classification outputs to new folders you will not overwrite regression outputs. If you replace `scalers.json` or other global artifacts, be careful — but the label creation step does not change scalers (unless you re-save scalers differently). I recommended saving classification datasets separately.
 - **What about samples lost due to NaN labels?**
You will lose windows whose `toxin_class` is NaN (i.e., the target value is missing). That's correct and expected — you cannot train on unknown labels.
-

9) Example: end-to-end small snippet (classification part only)

This is a minimal assembly showing the key inserts (not the whole file):

```
# after df_all creation and cleaning, before normalization or
right after:
THRESHOLD = 160.0
df_all['toxin_class'] = np.where(df_all['dsp_toxins'] == -1,
np.nan, (df_all['dsp_toxins'] > THRESHOLD).astype(int))

# keep original normalization step
df_all, scalers = global_normalize(df_all, EXPECTED_COLS)

# call make_supervised for classification
Xc, yc, dates_c, site_ids_c, splits_c = make_supervised(
    df_all, feats, target='dsp_toxins',
    time_steps=TIME_STEPS, horizon=FORECAST_HORIZON,
    class_target='toxin_class', class_mode='single'
)

# filter
Xc, yc, dates_c = filter_valid_class_samples(Xc, yc, dates_c)
yc = yc.reshape(-1, 1).astype(int) # ensure shape (N,1)

# save to separate folder
```



```
save_datasets(Xc, yc, dates_c, site_ids_c, splits_c,  
os.path.join(OUTPUT_DIR, variant + "_classification"))
```

10) Final notes / best practices

- Start with **single-horizon classification**. Train and debug that first.
 - Monitor class balance (fraction of positives). If positives < ~5–10% you must address imbalance (class weights, oversampling, focal loss).
 - Keep regression pipeline intact so you can compare: classification gives a different task and different evaluation metrics.
 - For confusion matrices, compute them per split **only** on the rows you included (after dropping NaN labels). That will explain why confusion matrix counts differ from total windows prior to filtering.
-

If you want, I can now:

- produce an exact **modified copy** of your preprocessing script with the classification additions included (single-horizon), **or**
- produce the **minimal model changes** to train a binary LSTM (sigmoid output, BCE, class weights), and an evaluation snippet to compute confusion matrices per split.

Which would you like next?

User Prompt:

I have the additional question as to what is the difference between the confusion matrix created at the end of the original model and what would happen with classification model

ChatGPT said:

Excellent question — and it's one of the most important conceptual distinctions between your **current regression confusion matrices** and what you'll get once you switch to a **true classification model**.

Let's go step by step — I'll explain both mathematically and practically what's going on, so you'll see why the two "confusion matrices" actually measure very different things.

✖ 1. What your current confusion matrix actually represents (regression version)

Right now, you're running a **regression model**, meaning your LSTM outputs a **continuous numeric prediction** (e.g., toxin concentration in $\mu\text{g/kg}$).

The code that produces a confusion matrix at the end probably does something like this:

```
exceed_true = (y_true > 160)
exceed_pred = (y_pred > 160)
cm = confusion_matrix(exceed_true, exceed_pred)
```

So what you're really doing is **converting a regression output into a binary exceedance prediction *after the fact***.

That means:

- The model never directly learned to classify — it learned to **predict a real number** (e.g., 125.4 $\mu\text{g/kg}$).
- You then manually binarized that continuous output around 160 $\mu\text{g/kg}$ to form "exceed" vs "non-exceed".

What this confusion matrix measures:

It shows **how often the model's predicted toxin concentration crosses the regulatory threshold correctly**, i.e.:

	True $\leq$ 160	True $>$ 160
Pred $\leq$ 160	TN	FN
Pred $>$ 160	FP	TP

But note:

- The model's decision boundary is *implicit* — it's not optimized to minimize classification error, it's optimized for **RMSE or MAE** (regression losses).
- That means your confusion matrix is *post hoc* — it tells you how well your regression approximations happen to separate exceedances, but the model itself

didn't try to make that separation.

2. What a classification model's confusion matrix represents

Once you switch to a classification setup:

- The model directly predicts **P(exceedance)** — a probability between 0 and 1 (via sigmoid activation).
- The loss function (binary crossentropy) explicitly trains it to correctly separate 0s and 1s.
- The confusion matrix now evaluates the model **on the task it was trained for**.

So the confusion matrix now truly measures classification accuracy:

- **TP**: model predicted "exceed" and it was indeed above 160
- **TN**: model predicted "non-exceed" and it was below 160
- **FP**: model said "exceed" when it wasn't
- **FN**: model said "non-exceed" when it was

Mathematically:

If the model output = $\text{sigmoid}(z)$ = probability of exceedance

Then you threshold it (e.g. $p > 0.5 \rightarrow \text{class } 1$):

```
y_pred_class = (model.predict(X_test) > 0.5).astype(int)
cm = confusion_matrix(y_true, y_pred_class)
```

This confusion matrix is **fundamental to the model's objective**, not an afterthought like before.

3. Key conceptual differences

Aspect	Regression model confusion matrix	Classification model confusion matrix
--------	-----------------------------------	---------------------------------------

Model output	Continuous (e.g. 0–1 normalized toxin value)	Probability (0–1) of exceedance
Training goal	Minimize numeric error (MAE, MSE)	Minimize classification error (cross-entropy)
Threshold	Arbitrary — imposed after training (e.g., 160 µg/kg)	Built-in — learned during training via logits
Interpretation	How often predicted toxin levels fall on correct side of 160	How often model correctly predicts exceed vs non-exceed
What it tests	Indirect classification ability of regression output	Direct classification accuracy, recall, precision
Best evaluation metrics	RMSE, MAE, R ²	Accuracy, Precision, Recall, F1, AUC

4. Numerical example to visualize the difference

Suppose a few samples:

True toxin (µg/kg)	Predicted toxin	True exceed?	Pred exceed?
180	120	1	0
30	50	0	0
170	165	1	1
90	170	0	1

→ Regression confusion matrix (using 160 threshold):

- TP = 1 (170 predicted 165)
- TN = 1 (30 predicted 50)
- FP = 1 (90 predicted 170)
- FN = 1 (180 predicted 120)

Now if you trained a **classifier**, it might output:

True exceed?	Predicted probability	Predicted class (0.5 threshold)
1	0.75	1
0	0.10	0
1	0.65	1
0	0.85	1

Here the model *knows* it's a binary problem. The confusion matrix reflects its true classification performance.

You can tune the 0.5 threshold to improve recall or precision (something you can't meaningfully do with regression outputs without remapping them).

5. Practical consequences for your work

Goal	Use Regression Matrix	Use Classification Matrix
Predict <i>how much</i> toxin	✓ Best choice	✗ Not applicable
Predict <i>whether limit is exceeded</i>	⚠ Works, but suboptimal	✓ Ideal
Compute RMSE/MAE metrics	✓	✗ Not defined
Compute F1/Precision/Recall	⚠ Post-hoc only	✓ True metrics
Tune model to improve exceedance detection	✗ Not aligned with loss	✓ Perfectly aligned

So if your main scientific or management question is:

“Can we predict whether toxin levels will exceed 160 µg/kg next week?”

then **classification is the correct formulation** — and your confusion matrix now has true diagnostic meaning.

6. TL;DR Summary

Feature	Regression CM	Classification CM
---------	---------------	-------------------

Model output	continuous numeric toxin level	probability of exceedance
Label type	continuous value (scaled)	binary (0/1)
Loss function	MSE / MAE	Binary Crossentropy
CM threshold	manually applied (e.g. 160 µg/kg)	0.5 (can tune)
Interpretation	post hoc measure of threshold accuracy	true measure of model skill
Best use case	quantitative forecasting	event prediction

Would you like me to show you **exactly how to compute both side by side** (regression-based and classification-based confusion matrices) so you can compare them numerically on your data?

That's usually the best way to see how the interpretations differ in practice.

User Prompt:

ok so how would i generate a pre_processing script for the classification model

ChatGPT said:

Perfect — this is a **really smart next step**, because once you move from *regression* (predicting toxin concentration) to *classification* (predicting exceedance yes/no), your **preprocessing must change slightly**, especially in how it generates your targets (*y*).

Let's go step-by-step through exactly what you need to modify and *why*, and then I'll show you a full working **classification-ready preprocessing script** at the end.

1. What stays the same

You **still need** the same basic preprocessing steps:

- Load and merge sites
- Resample weekly
- Handle missing values

- Normalize numeric inputs
- Create supervised windows (X) with temporal context

Because your classification model still needs the **same temporal patterns** as input — it just has a **different target**.

2. What changes conceptually

 **Old (regression) target:**

```
y = [dsp_toxins at t+1 ... t+4]
```

The model learned to predict *continuous values* (e.g., 83.5 µg/kg).

 **New (classification) target:**

```
y = [1 if dsp_toxins > 160 else 0]
```

We instead train the model to output a **binary sequence** for the next 4 weeks, saying whether toxin levels exceed the 160 µg/kg regulatory limit.

Optional simplification

You can also choose to predict **only one binary label per window**, for example:

“Will the level exceed 160 µg/kg *at least once* in the next 4 weeks?”

That depends on whether you want:

- **multi-step classification** (4 outputs per window → one for each week)
- or **single-step classification** (1 output per window → aggregate exceedance)

For consistency with your current LSTM structure (**horizon=4**), I'll show the **multi-step version**.

3. Implementation details to modify

Here's what changes line by line in your current preprocessing:

(a) Keep your input window creation (X) the same

Because the past 12 weeks of features are still your input.

```
X_all.append(data[i:i + time_steps, :len(features)])
```

(b) Change your y target creation

Instead of raw toxin values:

```
y_all.append(data[i + time_steps:i + time_steps + horizon,  
target_idx])
```

You'll binarize it relative to 160 µg/kg **before normalization**, like so:

```
# Convert to binary exceedance: 1 if toxin > 160, else 0  
y_window = (data[i + time_steps:i + time_steps + horizon,  
target_idx] > 160).astype(int)  
y_all.append(y_window)
```

(c) Normalization

You still normalize your **features**, but **do NOT normalize** your target anymore (it's now 0/1).

So your global normalization loop stays the same — it only affects inputs, not the binary targets.

4. Important note on data leakage

Make sure you **binarize after clipping and interpolation** — i.e., once the toxin column is fully cleaned, but **before scaling** to [0,1].

Otherwise, your threshold (160 µg/kg) becomes meaningless after normalization.

5. Full modified script

Here's a complete, classification-ready preprocessing script adapted from your regression one (changes annotated):


```

"""
Preprocessing for LSTM Classification Model (DSP Exceedance
Prediction)
-----
-----
Same structure as regression preprocessing, but targets (y) are
now
binary exceedance indicators: 1 if dsp_toxins > 160 µg/kg, else
0.
"""

import pandas as pd
import numpy as np
import glob
import os
import json

# ===== CONFIG ===== #
INPUT_PATH = "all_sites/*.xlsx"
OUTPUT_DIR = "preprocessed_classification"
os.makedirs(OUTPUT_DIR, exist_ok=True)

SITES_USED = ["RIAV1", "L5b", "ETJ1", "LAL", "L7c1", "LAG",
               "POR2", "FAR1"]

EXPECTED_COLS = [
    "dsp_toxins", "dsp_phyto", "mean_sst", "mean_chl",
    "mean_temp",
    "max_temp", "min_temp", "mean_wind_intensity", "wind_dir",
    "rainfall"
]

TIME_STEPS = 12
FORECAST_HORIZON = 4
REG_LIMIT = 160.0
# ===== #

def load_and_merge_sites(input_path):
    all_dfs = []
    for file in glob.glob(input_path):

```

```

    site_code = os.path.basename(file).split("_")[0]
    if site_code not in SITES_USED:
        continue

    df = pd.read_excel(file,
parse_dates=["Date"]).set_index("Date").sort_index()
    df = df.resample("W-WED").mean()

    for col in EXPECTED_COLS:
        if col not in df.columns:
            df[col] = np.nan

    df["dsp_toxins"] = df["dsp_toxins"].clip(lower=0)

    missing_frac = df[EXPECTED_COLS].isna().mean()
    for col in EXPECTED_COLS:
        if missing_frac[col] <= 0.25:
            df[col] = df[col].interpolate(method="time",
limit_direction="both")
        else:
            df[col] = df[col].fillna(-1)

    df["Site"] = site_code
    all_dfs.append(df)

df_all = pd.concat(all_dfs).sort_index()
df_all[EXPECTED_COLS] = df_all[EXPECTED_COLS].fillna(-1)
return df_all

def global_normalize(df, cols):
    scalars = {}
    for col in cols:
        valid_vals = df.loc[df[col] != -1, col]
        if valid_vals.empty:
            col_min, col_max = 0.0, 1.0
        else:
            col_min, col_max = valid_vals.min(),
valid_vals.max()

```

```

        scalers[col] = {"min": float(col_min), "max":
float(col_max)}

        if col != "dsp_toxins": #  Don't normalize the
classification target
            if col_max > col_min:
                df[col] = np.where(
                    df[col] == -1,
                    -1,
                    (df[col] - col_min) / (col_max - col_min)
                )
            else:
                df[col] = 0.0
    return df, scalers

def chronological_split(df):
    start_date, end_date = df.index.min(), df.index.max()
    train_weeks = int(2.5 * 52)
    test_weeks = 52
    val_weeks = 293 - (train_weeks + test_weeks)

    train_end = start_date + pd.Timedelta(weeks=train_weeks)
    val_end = train_end + pd.Timedelta(weeks=val_weeks)

    def get_split(date):
        if date <= train_end:
            return "train"
        elif date <= val_end:
            return "val"
        else:
            return "test"

    df["split"] = df.index.to_series().apply(get_split)
    return df

def make_supervised(df, features, target="dsp_toxins",
time_steps=12, horizon=4):
    """Create supervised samples per site (classification
version)."""

```

```

X_all, y_all, dates_all, site_ids_all, splits_all = [], [],
[], [], []

for site, group in df.groupby("Site"):
    group = group.sort_index()
    data = group[features + [target]].values
    target_idx = group.columns.get_loc(target)
    dates = group.index.values
    split_col = group["split"].values

    for i in range(len(group) - time_steps - horizon):
        X_all.append(data[i:i + time_steps, :len(features)])

        # ♦ Classification: binary exceedance label
        y_window = (data[i + time_steps:i + time_steps +
horizon, target_idx] > REG_LIMIT).astype(int)
        y_all.append(y_window)

        dates_all.append(dates[i + time_steps])
        site_ids_all.append(site)
        splits_all.append(split_col[i + time_steps])

    return np.array(X_all), np.array(y_all),
np.array(dates_all), np.array(site_ids_all),
np.array(splits_all)

def save_datasets(X, y, dates, site_ids, splits, output_path):
    os.makedirs(output_path, exist_ok=True)
    for s in ["train", "val", "test"]:
        idx = splits == s
        np.save(os.path.join(output_path, f"X_{s}.npy"), X[idx])
        np.save(os.path.join(output_path, f"y_{s}.npy"), y[idx])
        np.save(os.path.join(output_path, f"dates_{s}.npy"),
dates[idx])
        np.save(os.path.join(output_path, f"site_ids_{s}.npy"),
site_ids[idx])

# ===== PIPELINE ===== #

print("\nLoading and merging all sites...")

```

```

df_all = load_and_merge_sites(INPUT_PATH)
print(f"Loaded merged dataset with shape: {df_all.shape}")

print("\nNormalizing globally...")
df_all, scalars = global_normalize(df_all, EXPECTED_COLS)

print("\nApplying chronological split...")
df_all = chronological_split(df_all)

df_all.to_csv(os.path.join(OUTPUT_DIR,
"merged_cleaned_classification.csv"))
with open(os.path.join(OUTPUT_DIR, "scalars.json"), "w") as f:
    json.dump(scalars, f, indent=4)

print("\nCreating supervised datasets (12→4 classification)...")
VARIANTS = {
    "univariate": ["dsp_toxins"],
    "bivariate": ["dsp_toxins", "dsp_phyto"],
    "multivariate": EXPECTED_COLS
}

for variant, feats in VARIANTS.items():
    print(f"  Processing {variant}...")
    X, y, dates, site_ids, splits = make_supervised(
        df_all, feats, target="dsp_toxins",
        time_steps=TIME_STEPS, horizon=FORECAST_HORIZON
    )

    print(f"  Total samples: {len(X)} ({np.sum(splits=='train')}
train, "
        f"{np.sum(splits=='val')} val,
{np.sum(splits=='test')} test)")

    save_datasets(X, y, dates, site_ids, splits,
        os.path.join(OUTPUT_DIR, variant))

print("\n✅ Classification preprocessing complete.")
print("Binary exceedance targets (0=below, 1=above 160 µg/kg)")
print("Datasets saved in:", OUTPUT_DIR)

```

✓ 6. Summary of Key Changes

Step	Old (Regression)	New (Classification)	Why
Target values	Continuous DSP toxin levels	0/1 (exceedance)	Model learns event probability, not concentration
Normalization	Apply to all features	Skip <code>dsp_toxins</code>	Binary label shouldn't be scaled
Threshold	Applied after training	Applied during preprocessing	Needed to define classification labels
Model output	Numeric (float)	Probability (sigmoid)	Enables confusion matrix + crossentropy loss
Evaluation	MAE, RMSE, R ²	Accuracy, Recall, F1, AUC	Classification metrics instead of regression ones

Would you like me to show the **matching LSTM model and training code** (classification version with binary crossentropy and sigmoid outputs) that fits the outputs of this new preprocessing pipeline?
That'll be your next logical step.

User Prompt:

explain this list comprehension `y_window = (data[i + time_steps:i + time_steps + horizon, target_idx] > REG_LIMIT).astype(int)`

ChatGPT said:

Excellent — that's a really good line to zoom in on, because it's **compact but does a lot of work**.

Let's break it down piece by piece so you understand exactly what's happening and *why*.

The full line

```
y_window = (data[i + time_steps:i + time_steps + horizon,
target_idx] > REG_LIMIT).astype(int)
```

✖ Context

This line sits inside the **windowing loop** of the preprocessing function:

```
for i in range(len(group) - time_steps - horizon):  
    ...  
    y_window = (data[i + time_steps:i + time_steps + horizon,  
target_idx] > REG_LIMIT).astype(int)
```

At this point in the loop:

- `data` = NumPy array of your site's values for all features + target.
It's shape looks like `(num_weeks, num_features + 1)`
- `target_idx` = column index of "dsp_toxins" in that array.
So `data[:, target_idx]` is the time series of toxin levels for that site.
- `time_steps` = how many **past weeks** go into one input window (`X`)
→ e.g. 12 weeks of history.
- `horizon` = how many **future weeks** you want to predict (`y`)
→ e.g. 4 weeks ahead.

Step-by-step expansion

1 `data[i + time_steps:i + time_steps + horizon, target_idx]`

This extracts a *slice* of the target column (`dsp_toxins`) for the **future horizon** you're trying to predict.

If:

- `i = 10`
- `time_steps = 12`
- `horizon = 4`

Then this line gets:

```
data[22:26, target_idx]
```

That's the toxin levels for **weeks 22, 23, 24, 25**, which correspond to **the next 4 weeks after your 12-week input window**.

Example:

```
data[22:26, target_idx] = [85.0, 174.0, 210.5, 140.0]
```

❷ `( ... > REG_LIMIT)`

`REG_LIMIT` is defined earlier as:

```
REG_LIMIT = 160.0
```

So this comparison checks whether each toxin level **exceeds the regulatory limit**.

Applied to our example:

```
[85.0, 174.0, 210.5, 140.0] > 160.0
```

→ gives the boolean array:

```
[False, True, True, False]
```

❸ `.astype(int)`

This converts the boolean array into integers:

- `False` → `0`
- `True` → `1`

So now:

```
[False, True, True, False].astype(int)
```

becomes:

```
[0, 1, 1, 0]
```

✓ Final result: `y_window`

Now you have a **binary label vector** for that window, one value per horizon step:

Week offset	Toxin level	Exceeds 160?	Label
t+1	85.0	No	0
t+2	174.0	Yes	1
t+3	210.5	Yes	1
t+4	140.0	No	0

So:

```
y_window = np.array([0, 1, 1, 0])
```

📦 Why we do this

Because we want the model to learn:

“Given the last 12 weeks of data, what is the probability that toxin levels will exceed 160 µg/kg in each of the next 4 weeks?”

This lets you build a **multi-step classification** model — the output layer will have 4 neurons, each representing one week’s exceedance prediction.

🧠 TL;DR Summary

Part	Meaning
<code>data[i + time_steps:i + time_steps + horizon, target_idx]</code>	Future toxin values for horizon window
<code>> REG_LIMIT</code>	Compare each value to 160 µg/kg
<code>.astype(int)</code>	Convert booleans (True/False) → 1/0

✓ Result

Binary label vector (shape = [horizon])
for classification target

Would you like me to show how that binary label looks for an **entire dataset** (e.g., what shape **y** ends up with and how it lines up with the **X** windows)? That makes the concept even clearer visually.

User Prompt:

ok so now how would i create a classification model

ChatGPT said:

Perfect — now that your **preprocessing** logic is clear (and you understand that we're converting toxin levels into 0/1 exceedances), the next step is to actually build a **classification model** that predicts these binary outcomes.

Let's go step-by-step so you understand exactly **what changes** are needed compared to the regression model.

🧩 1. What changes conceptually

Your original **regression model**:

- Input: **X** → last 12 weeks of features.
- Output: **y** → 4 continuous toxin values (next 4 weeks).

Now for **classification**:

- Input: **exactly the same X**.
- Output: **y** → 4 binary labels (e.g., **[0, 1, 0, 0]**) indicating whether each future week exceeds 160 µg/kg.

So you're just changing:

The **nature of the target** and the **output layer + loss function** in your network.

⚙️ 2. Preprocessing adjustments

You already have this in place (based on your last message):

```
y_window = (data[i + time_steps:i + time_steps + horizon,
target_idx] > REG_LIMIT).astype(int)
```

That line turns the continuous toxin values into binary (0 or 1) targets.

So the only other changes in preprocessing are:

- ✓ Make sure you save **y** arrays as integers (they will be automatically from `.astype(int)`).
- ✓ No scaling of **y** needed (we don't normalize labels).

That's it — the input normalization stays the same.

🧠 3. Model architecture changes (LSTM classifier)

Here's a full Keras example for the **classification** version of your LSTM:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam

# Example input shape (should match your preprocessing)
# X shape: (samples, 12, num_features)
# y shape: (samples, 4)
time_steps = 12
num_features = 10 # depends on variant
horizon = 4

model = Sequential([
    LSTM(64, input_shape=(time_steps, num_features),
return_sequences=False),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dense(horizon, activation='sigmoid') # <- for binary output
per horizon step
])
```

```
# For multi-label binary classification (4 steps ahead)
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy', # <- classification loss
    metrics=['accuracy']
)

model.summary()
```



4. Training

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=50,
    batch_size=32,
    verbose=1
)
```

Each target `y_train[i]` is a **vector of 4 binary values**, one per horizon step.
So the model will try to learn the probability of exceedance at each horizon point.



5. Evaluation and confusion matrices

After training, predictions will look like this:

```
y_pred = model.predict(X_test) # shape: (num_samples, 4)
```

These are **probabilities**, so you convert them to 0/1:

```
y_pred_bin = (y_pred > 0.5).astype(int)
```

You can then generate confusion matrices **per horizon**, for example:

```
from sklearn.metrics import confusion_matrix,
classification_report
```

```

for h in range(horizon):
    cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
    print(f"\nConfusion matrix for horizon t+{h+1}")
    print(cm)
    print(classification_report(y_test[:, h], y_pred_bin[:, h]))

```

This tells you how well the model classifies exceedances each week ahead (t+1, t+2, etc.).

✂ 6. Notes on architecture variations

Depending on your dataset size, you can experiment with:

- Adding more LSTM layers (stacked LSTM)
- Using `return_sequences=True` and a final `TimeDistributed(Dense(1, activation='sigmoid'))`
- Adding regularization (Dropout, L2)
- Using class weights if exceedances are rare (imbalanced data)

For example, a time-distributed architecture:

```

from tensorflow.keras.layers import TimeDistributed, RepeatVector

model = Sequential([
    LSTM(64, input_shape=(time_steps, num_features)),
    RepeatVector(horizon),
    LSTM(64, return_sequences=True),
    TimeDistributed(Dense(1, activation='sigmoid'))
])

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

```

This one outputs `(samples, horizon, 1)` — good for sequence-to-sequence classification.

✓ 7. Summary: Key Differences from Regression

Aspect	Regression	Classification
y values	Continuous toxin levels	Binary exceedance (0/1)
Output layer	<code>Dense(horizon, activation='linear')</code>	<code>Dense(horizon, activation='sigmoid')</code>
Loss function	<code>mse</code> or <code>mae</code>	<code>binary_crossentropy</code>
Evaluation	RMSE, MAE, R^2	Accuracy, precision, recall, F1, confusion matrix
Postprocessing	Rescale predicted values	Threshold probabilities ($0.5 \rightarrow 1/0$)

Would you like me to generate a **full ready-to-run script** (loading your existing preprocessed `.npy` data and training a classification LSTM from scratch)?
I can include training, validation, and per-horizon confusion matrices.

User Prompt:

so what would that full script look like

ChatGPT said:

Perfect — let's build the **full classification LSTM script** end to end.

This will take your preprocessed `.npy` files (from your modified preprocessing script) and train a model to predict **whether DSP toxin levels exceed 160 µg/kg** for the next 4 weeks.

Overview

This script:

1. Loads the preprocessed datasets (`X_train.npy`, `y_train.npy`, etc.)

2. Builds and trains an **LSTM classifier**
3. Evaluates accuracy and confusion matrices per horizon step
4. Saves model + results

It assumes:

- The preprocessing step already converted **y** into 0/1 exceedances.
- The dataset structure is the same as before (e.g., multivariate, univariate, etc.).

🔗 Full Script — `run_lstm_classification.py`

```
"""
LSTM Classification Model for DSP Toxin Exceedance Prediction
-----
Predicts whether DSP toxin levels exceed the regulatory limit
(160 µg/kg)
for the next 4 weeks, based on the past 12 weeks of features.

Input: X_[split].npy (shape: samples, 12, num_features)
Target: y_[split].npy (shape: samples, 4) with binary values
(0/1)
"""

import numpy as np
import os
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import confusion_matrix,
classification_report

# =====
# --- Configuration ---
# =====
VARIANT = "multivariate" # or 'univariate', 'bivariate'
```

```

DATA_DIR = f"preprocessed/{VARIANT}"
OUTPUT_DIR = f"results/{VARIANT}_classification"
os.makedirs(OUTPUT_DIR, exist_ok=True)

TIME_STEPS = 12
HORIZON = 4
EPOCHS = 100
BATCH_SIZE = 32
LR = 0.001
REG_LIMIT = 160 # µg/kg threshold (for reference only)

# =====
# --- Load Datasets ---
# =====
print(f"\n📁 Loading preprocessed data from {DATA_DIR}...")

X_train = np.load(os.path.join(DATA_DIR, "X_train.npy"))
y_train = np.load(os.path.join(DATA_DIR, "y_train.npy"))
X_val    = np.load(os.path.join(DATA_DIR, "X_val.npy"))
y_val    = np.load(os.path.join(DATA_DIR, "y_val.npy"))
X_test   = np.load(os.path.join(DATA_DIR, "X_test.npy"))
y_test   = np.load(os.path.join(DATA_DIR, "y_test.npy"))

print(f"Shapes -> X_train: {X_train.shape}, y_train: {y_train.shape}")
num_features = X_train.shape[2]

# =====
# --- Build LSTM Classification Model ---
# =====
def build_lstm_classifier(input_shape, horizon):
    model = Sequential([
        LSTM(64, input_shape=input_shape,
return_sequences=False),
        Dropout(0.3),
        Dense(32, activation='relu'),
        Dense(horizon, activation='sigmoid') # 4 binary outputs
(0-1)
    ])
    model.compile(

```



```

        optimizer=Adam(learning_rate=LR),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model

model = build_lstm_classifier((TIME_STEPS, num_features),
HORIZON)
model.summary()

# =====
# --- Train Model ---
# =====
print("\n🚀 Training model...")
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

# =====
# --- Evaluate ---
# =====
print("\n📈 Evaluating on test set...")
y_pred = model.predict(X_test)
y_pred_bin = (y_pred > 0.5).astype(int)

# --- Per-horizon metrics ---
for h in range(HORIZON):
    print(f"\n=== Horizon t+{h+1} ===")
    cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
    print("Confusion Matrix:")
    print(cm)
    print("\nClassification Report:")
    print(classification_report(y_test[:, h], y_pred_bin[:, h],
digits=3))

# =====

```

```
# --- Save Model and Results ---
# =====
model.save(os.path.join(OUTPUT_DIR, "lstm_classifier.h5"))
np.save(os.path.join(OUTPUT_DIR, "y_pred.npy"), y_pred)
np.save(os.path.join(OUTPUT_DIR, "y_pred_bin.npy"), y_pred_bin)

print(f"\n✅ Model and predictions saved in {OUTPUT_DIR}")
```

🧩 Key Notes on the Script

Step	Description
Preprocessing assumption	You already converted toxin values into binary 0/1 exceedances in preprocessing.
Model	Uses a single LSTM layer + dropout + dense layers with sigmoid activation.
Loss function	<code>binary_crossentropy</code> — for classification.
Output shape	<code>(samples, 4)</code> → predicts 4 binary probabilities (t+1 ... t+4).
Evaluation	Prints confusion matrix + precision/recall/F1 per horizon.
Outputs saved	Model <code>.h5</code> + predictions <code>.npy</code> files.

⚙️ Optional improvements you can add later

Class imbalance handling (if exceedances are rare):

```
from sklearn.utils.class_weight import compute_class_weight
weights = compute_class_weight('balanced', classes=[0,1],
y=y_train.flatten())
class_weights = {0: weights[0], 1: weights[1]}
model.fit(..., class_weight=class_weights)
```

1.

ROC-AUC metrics:

```
from sklearn.metrics import roc_auc_score
roc = roc_auc_score(y_test.flatten(), y_pred.flatten())
```

- 2.
3. **TimeDistributed architecture** if you prefer a sequence-to-sequence structure.

Would you like me to modify this script to use the **TimeDistributed** version (so the model outputs 4 time steps explicitly like a mini-sequence prediction)?

That version can sometimes perform better for horizon-based classification.

User Prompt: [copy of warning message from code]

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`. also i would like to plot the results, additionally why is this code so much shorter?

ChatGPT said:

Excellent questions — let's unpack this carefully.

 First: The HDF5 Warning

This line:

```
model.save("lstm_classifier.h5")
```

produces:

"You are saving your model as an HDF5 file ... use the native Keras format instead."

 Fix:

```
model.save("lstm_classifier.keras")
```

That's all — it's not an error, just a recommendation.

`.keras` is TensorFlow's **new native format**, safer and faster than `.h5`.

 Second: Why This Code Is Shorter

The **classification script is simpler** than your regression one because:

Concept	Regression Script	Classification Script
Scaling	Needed (to map toxin values 0–1, inverse transform predictions, etc.)	Not needed — already binary (0/1)
Inverse-scaling	Required before computing metrics	Not used
RMSE/MAE/R²	Computed per horizon	Replaced by accuracy, precision, recall
Continuous forecasts	Predicted numeric DSP levels	Predicts probabilities (sigmoid)
Confusion matrix thresholding	Done manually after inverse-scaling	Directly applied to predicted probabilities ($\geq 0.5 = 1$)

So all the extra complexity around scaling, clipping, and masking disappears. You're working purely in the **classification domain**, which is much more direct.

Third: Adding Plots of Results

Here's an upgraded version of the **evaluation section** that adds:

- Confusion matrices for each horizon
- Accuracy/recall/F1 plots
- Saves them in your output folder

Add this **after** model evaluation in your script:

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay,
accuracy_score, precision_score, recall_score, f1_score

print("\n Evaluating on test set (with plots)...")
y_pred = model.predict(X_test)
y_pred_bin = (y_pred > 0.5).astype(int)
```

```

accuracies, recalls, precisions, f1s = [], [], [], []

for h in range(HORIZON):
    print(f"\n=== Horizon t+{h+1} ===")
    y_true_h = y_test[:, h]
    y_pred_h = y_pred_bin[:, h]

    acc = accuracy_score(y_true_h, y_pred_h)
    prec = precision_score(y_true_h, y_pred_h, zero_division=0)
    rec = recall_score(y_true_h, y_pred_h, zero_division=0)
    f1 = f1_score(y_true_h, y_pred_h, zero_division=0)

    accuracies.append(acc)
    recalls.append(rec)
    precisions.append(prec)
    f1s.append(f1)

    print(f"Accuracy={acc:.3f}, Precision={prec:.3f},
Recall={rec:.3f}, F1={f1:.3f}")

    cm = confusion_matrix(y_true_h, y_pred_h, labels=[0,1])
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=["Below", "Above"])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f"Confusion Matrix - Horizon t+{h+1}")
    plt.tight_layout()
    plt.savefig(os.path.join(OUTPUT_DIR,
f"confusion_matrix_t+{h+1}.png"))
    plt.close()

# --- Plot metrics per horizon ---
plt.figure(figsize=(8,5))
plt.plot(range(1, HORIZON+1), accuracies, '-o',
label='Accuracy')
plt.plot(range(1, HORIZON+1), precisions, '-o',
label='Precision')
plt.plot(range(1, HORIZON+1), recalls, '-o', label='Recall')
plt.plot(range(1, HORIZON+1), f1s, '-o', label='F1-score')
plt.xlabel('Forecast Horizon (weeks ahead)')

```

```
plt.ylabel('Score')
plt.title('Classification Metrics by Forecast Horizon')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR,
"classification_metrics_by_horizon.png"), dpi=300)
plt.close()

print(f"\n✅ Confusion matrices and metrics plots saved in
{OUTPUT_DIR}")
```

🔍 Summary of Why the New Script Is Simpler

- **Regression** → **Classification** removes the need for numeric rescaling.
 - Predictions are **probabilities** directly mapped to 0/1.
 - No inverse transformations or continuous metrics.
 - Fewer dependencies (**r2_score**, **MAE**, etc.) required.
 - Cleaner evaluation pipeline.
-

Would you like me to extend this script to include **per-site classification metrics and confusion matrices** (similar to your regression script's per-site split evaluation)?

That would let you compare performance across different sites as before.

User Prompt:

what kind of parameters should be tested for this classification model

ChatGPT said:

Excellent — this is a key design question. When turning your **time series toxin forecasting problem** into a **binary classification task** (“is toxin ≥ 160 $\mu\text{g/L}$ or not?”), your parameter search needs to balance **temporal context**, **model capacity**, and **class imbalance**.

Let's break it down in detail 📌

🌱 1. Categories of Hyperparameters to Tune

You'll be testing parameters in **four main categories**:

Category	Examples	Why It Matters
A. Windowing / sequence setup	<code>time_steps</code> , <code>horizon</code>	Controls how much past data the model "sees." Too small = poor temporal learning; too large = overfitting / redundant info.
B. Model architecture	LSTM units, number of layers, dropout, dense size	Defines model capacity and regularization strength.
C. Optimization	Learning rate, batch size, optimizer	Determines convergence stability and training dynamics.
D. Class imbalance handling	Loss weighting, oversampling	Important because "above 160" events are likely much rarer than "below 160."

⚙️ 2. Key Parameters to Try

♦ (A) Window / Horizon

`time_steps` $\in$ {8, 12, 16, 24} # 8–24 weeks of lookback
`horizon` $\in$ {1, 2, 4} # predict 1–4 weeks ahead

- Shorter windows capture recent spikes.
- Longer windows capture seasonal or gradual buildup trends.

💡 In practice: 12–16 often balances signal & generalization well.

♦ (B) Model Architecture

`lstm_units` $\in$ {32, 64, 128}
`dense_units` $\in$ {32, 64}
`dropout` $\in$ {0.1, 0.2, 0.3}

`num_layers ∈ {1, 2}`

- **More LSTM units / layers** → higher capacity, but risk overfitting.
- **Dropout** combats overfitting.
- For small datasets (common in red tide work), 1–2 layers, 64 units each, and 0.2 dropout are solid baselines.

♦ **(C) Optimization Parameters**

`learning_rate ∈ {1e-3, 5e-4, 1e-4}`

`batch_size ∈ {16, 32, 64}`

`optimizer ∈ {'adam', 'rmsprop'}`

- Lower learning rate = smoother convergence, better generalization.
- `adam` is typically best for LSTMs, but `rmsprop` sometimes stabilizes sequences with noisy temporal variance.

♦ **(D) Class Imbalance Strategies**

Because most toxin levels will be *below* 160, your dataset will be imbalanced.

Try one of the following:

Option 1: Class Weights

Compute them automatically:

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np
```

```
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.array([0, 1]),
    y=y_train.flatten()
)
```



```
class_weights = dict(enumerate(class_weights))
```

Then pass:

```
model.fit(X_train, y_train, ..., class_weight=class_weights)
```

Option 2: Oversampling minority class

You can use **SMOTE** or **RandomOverSampler**, but note they're tricky for time series (they break temporal order).

For LSTM models, **class weighting** is generally safer.

3. Example of a Keras Tuner Search Space

Here's how you'd define this in a tuner (like `keras_tuner`):

```
def build_model(hp):
    model = Sequential()
    model.add(LSTM(
        units=hp.Choice("lstm_units", [32, 64, 128]),
        return_sequences=False,
        input_shape=(TIME_STEPS, num_features)
    ))
    model.add(Dropout(hp.Choice("dropout", [0.1, 0.2, 0.3])))
    model.add(Dense(
        hp.Choice("dense_units", [32, 64]),
        activation="relu"
    ))
    model.add(Dense(HORIZON, activation="sigmoid"))

    model.compile(
        optimizer=keras.optimizers.Adam(
            learning_rate=hp.Choice("learning_rate", [1e-3,
5e-4, 1e-4])
        ),
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )
    return model
```

Then you can run:

```
tuner = kt.RandomSearch(  
    build_model,  
    objective="val_accuracy",  
    max_trials=20,  
    directory="tuning_results",  
    project_name="lstm_classification"  
)
```

4. Metrics to Optimize

For imbalanced classification, accuracy can be misleading — you want:

- **F1-score** → balances precision & recall
- **Recall (Sensitivity)** → how many true “high toxin” events are detected
- **Precision** → how many predicted “high” events are actually high

So in your tuner, use:

```
objective=kt.Objective("val_f1_score", direction="max")
```

or compute F1 manually if needed.

Summary of Recommended Tuning Ranges

Parameter	Suggested Values	Comment
<code>time_steps</code>	8, 12, 16, 24	Lookback window
<code>horizon</code>	1, 2, 4	Forecast horizon
<code>lstm_units</code>	32, 64, 128	LSTM cell size

<code>dropout</code>	0.1–0.3	Regularization
<code>dense_units</code>	32, 64	Hidden layer size
<code>learning_rate</code>	1e-3–1e-4	LR search
<code>batch_size</code>	16–64	Training stability
<code>class_weight</code>	Balanced	Handle rare events
<code>optimizer</code>	adam, rmsprop	Training performance

If you'd like, I can generate a **full classification tuning script** (using Keras Tuner or manual grid search) that automatically tests these parameters, logs F1/accuracy, and saves the best model.

Would you like me to write that out next?

User Prompt:

what does the 0 represent does that mean predicted above or below the regulatory line and why are you recommending adam or rms prop if you said we had to use sigmoid before

ChatGPT said:

Excellent — this is a really insightful question 🙌

You've hit on **two key but easily confused points**:

- 1️⃣ **What does “0” vs “1” mean in classification output**
- 2️⃣ **Why we use `sigmoid` and optimizers like Adam/RMSprop — and how they're not the same thing**

Let's unpack each carefully.

1️⃣ What “0” and “1” Mean in Your Classification Model

In your classification setup, you're predicting **whether toxin $\geq$ regulatory limit (160 $\mu\text{g/L}$) or not.**

You **encode that condition** numerically when you create your target labels.
For example, in preprocessing you did something like:

```
y_window = (data[i + time_steps:i + time_steps + horizon,  
target_idx] > REG_LIMIT).astype(int)
```

That line means:

Condition	Label	Meaning
<code>toxin < 160</code>	0	Below limit (non-event, "safe")
<code>toxin ≥ 160</code>	1	Above limit (event, "unsafe")

So:

- 0 → below threshold
- 1 → above threshold

You can flip them if you prefer (it's arbitrary), but **you must be consistent** across training, evaluation, and confusion matrix plotting.

- ♦ The model learns to output a *probability* between 0 and 1 (via the sigmoid), which you then threshold at 0.5 to classify:

```
y_pred = (model.predict(X_test) > 0.5).astype(int)
```

⚙️ 2 Why We Use Sigmoid + Adam/RMSprop

These serve **very different roles** in your neural network:

Component	Purpose	Where It's Used
Sigmoid activation	Converts the model's final numeric output (logit) into a probability between 0 and 1.	Last layer of the neural net

**Optimizer
(Adam /
RMSprop)**

Controls *how* the model
updates its weights during
training.

Compiles/training phase
(`model.compile(optimizer=..
..)`)

So they're not interchangeable at all — you use **both**.

♦ How They Work Together

Final layer:

```
model.add(Dense(1, activation='sigmoid'))
```

- This says: “Output a probability of being class 1 (above 160).”
- Example: model outputs `0.85` → 85% chance toxin $\geq$ 160.

Optimizer choice:

```
model.compile(optimizer='adam', loss='binary_crossentropy',  
metrics=['accuracy'])
```

- Adam or RMSprop determine *how weights get updated* each iteration based on the loss gradient.
- For LSTMs and sequential data, these optimizers handle non-stationary signals and vanishing gradients better than plain SGD.

So:

- `sigmoid` handles the *shape* of the output (probability).
- `adam` or `rmsprop` handle *how the model learns*.

They complement each other — not replace each other.

⚙ Summary

Concept	Role	Why Used
Sigmoid	Converts raw score → probability (0–1).	Needed for binary classification output.
0 / 1 labels	Represent below vs above threshold.	0 = below limit, 1 = above limit.
Adam / RMSprop	Weight update algorithms.	Handle noisy temporal data efficiently.

If you want, I can show you how your **final classification LSTM model** should be compiled and trained with these components properly — including how it generates probabilities, thresholds them, and produces a confusion matrix that maps 0 ↔ 1 correctly.

Would you like me to generate that full integrated training + evaluation script next?

User Prompt:

so what kind of activation functions could i use because right now i am using relu, but would other good options be leaky-relu, or tanh

ChatGPT said:

Excellent — that's *exactly* the right line of thinking 👍

Let's go through this carefully — because the activation function you choose **changes how your LSTM learns** patterns in the time series and how the network behaves internally.

We'll cover:

- 1 **Where activation functions appear** in your classification model
 - 2 **What each one (ReLU, LeakyReLU, tanh, etc.) does**
 - 3 **When to prefer each** — especially for *time series* data like toxin levels
 - 4 **What the final layer should always use**
-

🧩 1 Where Activations Go

In an LSTM classification model, you'll usually have **two types of activations**:

1. **Hidden layer activations** (inside LSTM and Dense layers)
2. **Output layer activation** (at the very end)

Example:

```
model = Sequential([
    LSTM(64, activation='tanh', return_sequences=True),
    Dropout(0.2),
    LSTM(32, activation='tanh'),
    Dense(16, activation='relu'),
    Dense(1, activation='sigmoid') # Output: probability (0-1)
])
```

You can experiment with the **hidden layers** (ReLU, tanh, etc.), but the **output layer must use sigmoid** for binary classification.

⚙️ 2 The Major Activation Functions

Activation	Formula / Shape	Pros	Cons	Typical Use
ReLU	$\max(0, x)$	Simple, fast, avoids vanishing gradient	Can “die” (output 0 for all negative inputs)	Default for dense layers
LeakyReLU	x if $x > 0$ else $0.01x$	Fixes “dead ReLU” problem	Slightly slower, introduces small negative slope	Good for small datasets or noisy features
tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}} \rightarrow \text{range } [-1, 1]$	Smooth, zero-centered, stable for sequences	Can saturate (vanishing gradients)	Excellent for time series (LSTMs)
sigmoid	$1 / (1 + e^{-x}) \rightarrow \text{range } [0, 1]$	Probabilistic output	Saturates faster than tanh	Final binary output layer only
ELU / SELU	Exponential variants of ReLU	Maintain non-zero mean	Slightly more compute-heavy	Deep networks with normalization

🌀 3 For Time Series Data (like DSP toxins)

When dealing with **temporal** data, especially where:

- Inputs can be both positive and negative (e.g., standardized environmental data)
- You want smooth gradient flow through time

→ **tanh** almost always performs best **inside LSTM cells**.

That's because:

- LSTM gates already use **sigmoid** internally.
- The cell state benefits from tanh's symmetric range (-1 to 1), making it numerically stable.

Example

```
LSTM(64, activation='tanh', recurrent_activation='sigmoid')
```

This is actually Keras's **default** — and for good reason.

4 Recommended Setup for Your Classification Task

Given your DSP toxin dataset (environmental + biological time series):

Layer Type	Suggested Activation	Reason
LSTM layers	tanh	Stable gradients and expressive temporal encoding
Dense hidden layers	ReLU or LeakyReLU	Nonlinear mapping between sequence features
Output layer	sigmoid	Produces [0,1] probability of being above 160

Example Code

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout,
LeakyReLU
```



```

model = Sequential([
    LSTM(64, activation='tanh', return_sequences=True,
input_shape=(12, num_features)),
    Dropout(0.2),
    LSTM(32, activation='tanh'),
    Dense(16),
    LeakyReLU(alpha=0.1),
    Dense(1, activation='sigmoid') # binary classification
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

```

5 Quick Summary

Use Case	Best Activation	Why
Inside LSTM	tanh	Stable & balanced gradients
Dense hidden layers	ReLU or LeakyReLU	Efficient nonlinear transform
Final output	sigmoid	Needed for binary classification
Regression (if used later)	linear	Predicts continuous values

If you'd like, I can help you create a **KerasTuner setup** that tests combinations like:

- **tanh** vs **relu** vs **leakyrelu**
- Different hidden unit sizes
- Dropout rates

...so you can empirically find which works best for your classification dataset.
Would you like me to generate that hyperparameter tuning template next?

User Prompt:

so is this stating the activation function for each layer? LSTM(64, activation='tanh', return_sequences=True), Dropout(0.2), LSTM(32, activation='tanh'), Dense(16, activation='relu'), Dense(1, activation='sigmoid') # Output: probability (0–1)

ChatGPT said:

Exactly  — you've got it right.

That snippet is explicitly specifying the **activation function used by each layer** in your neural network.

Let's break it down layer by layer to make sure it's crystal clear 

Full Line-by-Line Explanation

LSTM(64, activation='tanh', return_sequences=True)

- **Layer type:** LSTM (recurrent layer that processes sequences)
- **Units:** 64 neurons (memory cells)
- **Activation:** 'tanh'
→ This controls how the *cell outputs* are squashed between -1 and 1. It helps stabilize training over time steps.
- **return_sequences=True:**
→ Tells the layer to output a sequence at every time step (needed if you have another LSTM afterward).

Dropout(0.2)

- Randomly “turns off” 20% of neurons during training to prevent overfitting.
- No activation function here — dropout just randomly zeroes outputs.

LSTM(32, activation='tanh')

- Another LSTM layer with 32 units.
- Again uses '`tanh`' as the activation (for stability in time series).
- **Note:** `return_sequences=False` by default here — so this one outputs only the *final* time step.

`Dense(16, activation='relu')`

- A fully connected (feed-forward) layer with 16 neurons.
- **Activation:** '`relu`' (Rectified Linear Unit)
 - Makes the model nonlinear and helps it learn complex patterns. It outputs `0` for negative values and the value itself for positives.

`Dense(1, activation='sigmoid')`

- **Output layer** for binary classification.
- Outputs a single probability between **0 and 1**, representing:
 - **Close to 0** → below the regulatory threshold (safe)
 - **Close to 1** → above the threshold (toxic)
- The `sigmoid` ensures outputs are valid probabilities.

 **In short:**

Layer	Activation	Purpose
	n	
LSTM (1st)	<code>tanh</code>	Temporal encoding of sequence data
LSTM (2nd)	<code>tanh</code>	Further sequence compression

Dense (hidden)	<code>relu</code>	Nonlinear mapping
Dense (output)	<code>sigmoid</code>	Probability of toxic event (0–1)

If you wanted, you could swap in **LeakyReLU** or **ELU** for that middle Dense layer — but the **output must always remain sigmoid** for binary classification.

Would you like me to show you what it looks like if you replaced the middle `relu` with **LeakyReLU** (and explain why it might improve performance)?

User Prompt:

so what about the loss what can i change about that

ChatGPT said:

Excellent question — the **loss function** is *crucial* in classification, because it defines **what kind of mistake the model “cares about.”**

Let’s walk through this carefully so you understand *why* each choice matters, and how it ties to your problem (toxic bloom prediction above/below 160 µg AO kg⁻¹).

⚙️ 1 Why loss functions matter

The loss tells your model *how wrong* it is — and in binary classification, it’s comparing:

- The predicted probability (from `sigmoid`, between 0 and 1)
- The true label (0 = below limit, 1 = above limit)

So if the model predicts `0.8` for a sample that’s truly 1, that’s a *small* error. But if it predicts `0.1` for a true 1, that’s a *large* error.

🌱 2 Common loss options for binary classification

Loss Function	When to Use	Explanation
---------------	-------------	-------------

binary_crossentropy	✅ Default, general-purpose	Measures how far predicted probabilities are from actual labels. Works best when data is balanced or moderately imbalanced.
focal_loss	For <i>highly imbalanced</i> datasets	Adds more weight to misclassified examples, especially when one class (e.g., toxic events) is rare. Very useful if you have many 0s and few 1s.
hinge or squared_hinge	For SVM-like margin classifiers	Works if you interpret labels as -1/+1 instead of 0/1. Less common in Keras classification models.
bce_with_logits (from_logits=True)	If you remove the sigmoid in the output	Combines sigmoid + BCE internally for better numerical stability. Only use if the final layer has no activation .

⚖️ 3 Recommended for your case (toxin threshold)

Since your task is:

“Predict whether DSP toxins exceed 160 (1) or not (0)”
and you likely have **class imbalance** (few “toxic” weeks), I recommend:

✅ Option A — Start simple

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', 'Precision', 'Recall']
)
```

✅ Option B — If your 1s are much rarer

Try *focal loss* (needs TensorFlow Addons):

```
from tensorflow_addons.losses import SigmoidFocalCrossEntropy

model.compile(
    optimizer='adam',
    loss=SigmoidFocalCrossEntropy(alpha=0.75, gamma=2.0),
    metrics=['accuracy', 'Precision', 'Recall']
)
```

)

This emphasizes rare “toxic” events — penalizing the model more when it misclassifies them.

🎯 4 Optional weighting trick

If you don’t want focal loss, you can do:

```
model.fit(X_train, y_train, class_weight={0:1.0, 1:5.0})
```

This multiplies the loss of each class to balance the dataset — useful if you have, say, 5× fewer toxic events.

🧠 TL;DR

Situation	Recommended Loss
Normal (roughly balanced)	<code>binary_crossentropy</code>
Rare positive events	<code>SigmoidFocalCrossEntropy</code> or <code>class_weight</code>
No sigmoid output	<code>BinaryCrossentropy(from_logits=True)</code>

If you show me how many “toxic” samples (above 160) vs. “non-toxic” ones you have in your dataset, I can tell you **exactly** whether you should stick with `binary_crossentropy` or switch to a focal loss or weighted scheme. Would you like me to show you how to check that quickly in your preprocessed data?

User Prompt: [copy of classification model code]

How do I add the y_pred vs the y_true """ LSTM Classification Model for DSP Toxin Exceedance Prediction ----- Predicts whether DSP toxin levels exceed the regulatory limit (160 µg/kg) for the next 4 weeks, based on the past 12 weeks of features. Input: X_[split].numpy (shape: samples, 12, num_features) Target: y_[split].numpy (shape: samples, 4) with binary values (0/1) """ import numpy as np import os import tensorflow as tf import matplotlib.pyplot as plt from

```

tensorflow.keras.models import Sequential from tensorflow.keras.layers import LSTM,
Dense, Dropout from tensorflow.keras.optimizers import Adam from sklearn.metrics
import confusion_matrix, classification_report #
===== # ---
Configuration --- #
===== VARIANT =
"univariate" # or 'univariate', 'bivariate' DATA_DIR =
f"../preprocessed_classification/{VARIANT}" OUTPUT_DIR =
f"results/{VARIANT}_classification" os.makedirs(OUTPUT_DIR, exist_ok=True)
TIME_STEPS = 12 HORIZON = 4 EPOCHS = 100 BATCH_SIZE = 32 LR = 0.001
REG_LIMIT = 160 # µg/kg threshold (for reference only) #
===== # --- Load
Datasets --- #
===== print(f"\n
Loading preprocessed data from {DATA_DIR}...") X_train =
np.load(os.path.join(DATA_DIR, "X_train.npy")) y_train =
np.load(os.path.join(DATA_DIR, "y_train.npy")) X_val = np.load(os.path.join(DATA_DIR,
"X_val.npy")) y_val = np.load(os.path.join(DATA_DIR, "y_val.npy")) X_test =
np.load(os.path.join(DATA_DIR, "X_test.npy")) y_test = np.load(os.path.join(DATA_DIR,
"y_test.npy")) print(f"Shapes -> X_train: {X_train.shape}, y_train: {y_train.shape}")
num_features = X_train.shape[2] #
===== # --- Build
LSTM Classification Model --- #
===== def
build_lstm_classifier(input_shape, horizon): model = Sequential([ LSTM(64,
input_shape=input_shape, return_sequences=False), Dropout(0.3), Dense(32,
activation='relu'), Dense(horizon, activation='sigmoid') # 4 binary outputs (0–1) ])
model.compile( optimizer=Adam(learning_rate=LR), loss='binary_crossentropy',
metrics=['accuracy'] ) return model model = build_lstm_classifier((TIME_STEPS,
num_features), HORIZON) model.summary() #
===== # --- Train
Model --- # =====
print("\n Training model...") history = model.fit( X_train, y_train, validation_data=(X_val,
y_val), epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1 ) #
===== # ---
Evaluate --- #
===== print("\n 
Evaluating on test set..." y_pred = model.predict(X_test) y_pred_bin = (y_pred >
0.5).astype(int) # --- Per-horizon metrics --- for h in range(HORIZON): print(f"\n===
Horizon t+{h+1} ===") cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
print("Confusion Matrix:") print(cm) print("\nClassification Report:")
print(classification_report(y_test[:, h], y_pred_bin[:, h], digits=3)) #
===== # --- Save
Model and Results --- #
=====
model.save(os.path.join(OUTPUT_DIR, "lstm_classifier.keras"))

```

```
np.save(os.path.join(OUTPUT_DIR, "y_pred.npy"), y_pred)
np.save(os.path.join(OUTPUT_DIR, "y_pred_bin.npy"), y_pred_bin) print(f"\n✅ Model
and predictions saved in {OUTPUT_DIR}") '
```

ChatGPT said:

Excellent — your script is already very solid 🙌

Right now it trains, evaluates, and prints confusion matrices & reports.

What's missing is a **visual comparison** of `y_pred` vs `y_true`, which helps you see how well the model tracks the true binary events.

Let's walk through **how** to add these plots and **why** they're done a certain way.

🔗 1 What “y_pred vs y_true” means in classification

Since your model outputs **binary predictions** (0 = below, 1 = above limit) for each of the 4 horizons, you can visualize:

- The **true exceedance events** (`y_true`)
- The **predicted exceedances** (`y_pred_bin`)

This lets you see how well the model identifies bloom events (1s).

You can also visualize **predicted probabilities** (`y_pred`) to see model confidence.

🔗 2 Add plotting code

Below your evaluation loop, insert this section:

```
# =====
# --- Plot Predictions vs True Values ---
# =====
print("\n📊 Plotting predicted vs true values...")

for h in range(HORIZON):
    plt.figure(figsize=(10, 4))

    plt.plot(y_test[:, h], label="True", marker='o',
linestyle='--', alpha=0.7)
```



```

plt.plot(y_pred[:, h], label="Predicted Prob.",
color='orange', linewidth=2)
plt.plot(y_pred_bin[:, h], label="Predicted Class",
color='red', linestyle=':', alpha=0.7)

plt.title(f"LSTM Classification – Horizon t+{h+1}")
plt.xlabel("Sample Index")
plt.ylabel("Exceedance (0=Below, 1=Above)")
plt.ylim(-0.1, 1.1)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR,
f"y_pred_vs_true_t{h+1}.png"))
plt.close()

```

3 Explanation

- `y_test[:, h]` → actual exceedance labels (0 or 1)
- `y_pred[:, h]` → predicted probability (0–1)
- `y_pred_bin[:, h]` → binary class after thresholding (>0.5)

The resulting plot (for each horizon) shows:

- Ground truth exceedances as dashed line (0/1)
- The model's *confidence* (orange curve)
- The final binary classification (red dots)

So you can tell whether the model was *confidently wrong* or *unsure but correct*.

4 Save all results cleanly

Since you already save predictions, the plots will now appear in:

```
results/univariate_classification/  
├─ lstm_classifier.keras  
├─ y_pred.npy  
├─ y_pred_bin.npy  
├─ y_pred_vs_true_t1.png  
├─ y_pred_vs_true_t2.png  
├─ y_pred_vs_true_t3.png  
└─ y_pred_vs_true_t4.png
```

✅ (Optional) Add an overall comparison

If you want one combined figure for all horizons:

```
plt.figure(figsize=(10, 6))  
for h in range(HORIZON):  
    plt.plot(y_test[:, h] + h*1.2, label=f"True t+{h+1}",  
            linestyle='--')  
    plt.plot(y_pred_bin[:, h] + h*1.2, label=f"Pred t+{h+1}",  
            alpha=0.7)  
plt.yticks([])  
plt.legend()  
plt.title("Predicted vs True Exceedance Across Horizons")  
plt.xlabel("Sample Index")  
plt.tight_layout()  
plt.savefig(os.path.join(OUTPUT_DIR, "y_pred_vs_true_all.png"))  
plt.close()
```

That gives you one stacked comparison showing how the model behaves over multiple horizons at once.

Would you like me to also show how to **plot ROC curves and AUC scores** for each horizon?

That's often the most informative visualization in classification problems like this.

User Prompt:

why are we thresholding it?

ChatGPT said:

Ah — this is an important conceptual point. Let me explain clearly.

1 The model outputs probabilities

Your LSTM classifier's **last layer** uses a **sigmoid activation**, so for each horizon it outputs a number between **0 and 1**:

- **0.0** → model is very confident the sample is below the regulatory limit
- **1.0** → model is very confident the sample is above the regulatory limit
- **0.3, 0.7** → model is uncertain but leaning toward below/above

This is **not yet a class label**; it's a probability.

2 Thresholding turns probabilities into class labels

To actually **classify** a sample as “exceeding” or “not exceeding,” we apply a threshold:

```
y_pred_bin = (y_pred > 0.5).astype(int)
```

- **>0.5** → class 1 (“above regulatory limit”)
- **≤0.5** → class 0 (“below regulatory limit”)

Now **y_pred_bin** is **binary**, matching the format of **y_test**, so you can:

- Compute **accuracy, confusion matrices, precision/recall**
 - Plot **predicted vs true class**
 - Evaluate the model as a standard classifier
-

3 Why 0.5 by default?

0.5 is the standard threshold for **balanced binary classification**, but it's **not fixed**:

- If false negatives are worse than false positives (e.g., missing a toxic bloom is very bad), you might lower the threshold to 0.3 to be more “sensitive.”
- If false positives are worse, you might raise it to 0.7.

You can even **tune the threshold** based on ROC curves or F1 score for your dataset.

4 Summary

- **Without thresholding** → you have probabilities, which are nice for plotting and ROC/AUC analysis
 - **With thresholding** → you get hard 0/1 predictions, which you need for confusion matrices and classification reports
-

If you want, I can show a **diagram of what this looks like over time**, showing probabilities versus thresholded predictions versus true labels — it makes the concept much more intuitive.

Do you want me to do that?

User Prompt:

ok well those plots are not insightful, how are plots for classification normally plotted i dont recall

ChatGPT said:

Ah, yes — for classification tasks, plotting raw predicted vs true values like in regression isn't very informative. Instead, there are several standard approaches to visualize classifier performance, especially for something like your LSTM predicting toxin exceedance:

1 Confusion Matrix

- This is the most straightforward: shows **true positives, true negatives, false positives, false negatives**.
- You can do this **per horizon** (t+1, t+2, ...) since you have multiple outputs.

- Example (already in your code):

```
from sklearn.metrics import ConfusionMatrixDisplay

for h in range(HORIZON):
    cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
    disp = ConfusionMatrixDisplay(cm, display_labels=["Below",
"Above"])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(f"Horizon t+{h+1} Confusion Matrix")
    plt.show()
```

- Insight: you can quickly see **where the model is making errors**.
-

2 ROC Curve (Receiver Operating Characteristic)

- Plots **True Positive Rate vs False Positive Rate** as you vary the probability threshold.
- Useful to assess **model discrimination**.
- Can calculate **AUC (Area Under Curve)** to quantify performance.

```
from sklearn.metrics import roc_curve, auc

for h in range(HORIZON):
    fpr, tpr, thresholds = roc_curve(y_test[:, h], y_pred[:, h])
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f't+{h+1} (AUC = {roc_auc:.2f})')
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves per Horizon")
plt.legend()
plt.show()
```

- Insight: shows how well your model separates above/below limit for each horizon.
-

3 Precision-Recall Curve

- Especially useful if your **positive class (exceedance) is rare**.
- Plots **Precision vs Recall** at different thresholds.

```
from sklearn.metrics import precision_recall_curve

for h in range(HORIZON):
    precision, recall, thresholds =
precision_recall_curve(y_test[:, h], y_pred[:, h])
    plt.plot(recall, precision, label=f't+{h+1}')
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.show()
```

- Insight: helps decide an optimal threshold for classification, especially if false negatives are costly.
-

4 Probability Histograms

- Plot **histograms of predicted probabilities** for the two true classes.
- Helps visualize **how confident your model is** for each class.

```
plt.hist(y_pred[y_test[:, h]==0, h], bins=20, alpha=0.5,
label="Below")
plt.hist(y_pred[y_test[:, h]==1, h], bins=20, alpha=0.5,
label="Above")
plt.xlabel("Predicted probability")
plt.ylabel("Count")
```

```
plt.title(f"Horizon t+{h+1} Probability Distribution")
plt.legend()
plt.show()
```

- Insight: see overlap between predicted probabilities for “above” and “below.”

✓ Summary:

For classification tasks, you usually **don’t plot predicted vs true values** like in regression. Instead, you use:

- Confusion matrices
- ROC / AUC
- Precision-Recall curves
- Probability distributions

These tell you **how well the model is separating classes**, and whether the threshold you chose makes sense.

If you want, I can rewrite your **evaluation code** to automatically generate all these plots for all horizons — that gives a complete classification assessment.

Do you want me to do that?

User Prompt: [copy of classification model code]

```
explain to me how this model works and what benefit would making the original
regression model into a classification model """ LSTM Classification Model for DSP
Toxin Exceedance Prediction ----- Predicts
whether DSP toxin levels exceed the regulatory limit (160 µg/kg) for the next 4 weeks,
based on the past 12 weeks of features. Input: X_[split].npz (shape: samples, 12,
num_features) Target: y_[split].npz (shape: samples, 4) with binary values (0/1) """
import numpy as np import os import tensorflow as tf import matplotlib.pyplot as plt from
tensorflow.keras.models import Sequential from tensorflow.keras.layers import LSTM,
Dense, Dropout from tensorflow.keras.optimizers import Adam from sklearn.metrics
import confusion_matrix, classification_report #
===== # ---
Configuration --- #
===== VARIANT =
```

```

"univariate" # or 'univariate', 'bivariate' DATA_DIR =
f"../preprocessed_classification/{VARIANT}" OUTPUT_DIR =
f"results/{VARIANT}_classification" os.makedirs(OUTPUT_DIR, exist_ok=True)
TIME_STEPS = 12 HORIZON = 4 EPOCHS = 100 BATCH_SIZE = 32 LR = 0.001
REG_LIMIT = 160 # µg/kg threshold (for reference only) #
===== # --- Load
Datasets --- #
===== print(f"\n
Loading preprocessed data from {DATA_DIR}...") X_train =
np.load(os.path.join(DATA_DIR, "X_train.npy")) y_train =
np.load(os.path.join(DATA_DIR, "y_train.npy")) X_val = np.load(os.path.join(DATA_DIR,
"X_val.npy")) y_val = np.load(os.path.join(DATA_DIR, "y_val.npy")) X_test =
np.load(os.path.join(DATA_DIR, "X_test.npy")) y_test = np.load(os.path.join(DATA_DIR,
"y_test.npy")) print(f"Shapes -> X_train: {X_train.shape}, y_train: {y_train.shape}")
num_features = X_train.shape[2] #
===== # --- Build
LSTM Classification Model --- #
===== def
build_lstm_classifier(input_shape, horizon): model = Sequential([ LSTM(64,
input_shape=input_shape, return_sequences=False), Dropout(0.3), Dense(32,
activation='relu'), Dense(horizon, activation='sigmoid') # 4 binary outputs (0-1) ])
model.compile( optimizer=Adam(learning_rate=LR), loss='binary_crossentropy',
metrics=['accuracy'] ) return model model = build_lstm_classifier((TIME_STEPS,
num_features), HORIZON) model.summary() #
===== # --- Train
Model --- # =====
print("\n Training model...") history = model.fit( X_train, y_train, validation_data=(X_val,
y_val), epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1 ) #
===== # ---
Evaluate --- #
===== print(f"\n 
Evaluating on test set...") y_pred = model.predict(X_test) y_pred_bin = (y_pred >
0.5).astype(int) # --- Per-horizon metrics --- for h in range(HORIZON): print(f"\n===
Horizon t+{h+1} ===") cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
print("Confusion Matrix:") print(cm) print("\nClassification Report:")
print(classification_report(y_test[:, h], y_pred_bin[:, h], digits=3)) #
===== # --- Save
Model and Results --- #
=====
model.save(os.path.join(OUTPUT_DIR, "lstm_classifier.keras"))
np.save(os.path.join(OUTPUT_DIR, "y_pred.npy"), y_pred)
np.save(os.path.join(OUTPUT_DIR, "y_pred_bin.npy"), y_pred_bin) print(f"\n  Model
and predictions saved in {OUTPUT_DIR}") print(f"\n  Plotting predicted vs true
values...") for h in range(HORIZON): plt.figure(figsize=(10, 4)) plt.plot(y_test[:, h],
label="True", marker='o', linestyle='--', alpha=0.7) plt.plot(y_pred[:, h], label="Predicted
Prob.", color='orange', linewidth=2) plt.plot(y_pred_bin[:, h], label="Predicted Class",

```



```
color='red', linestyle=':', alpha=0.7) plt.title(f"LSTM Classification — Horizon t+{h+1}")
plt.xlabel("Sample Index") plt.ylabel("Exceedance (0=Below, 1=Above)") plt.ylim(-0.1,
1.1) plt.legend() plt.grid(True, linestyle='--', alpha=0.4) plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, f"y_pred_vs_true_{h+1}.png")) plt.close()
```

ChatGPT said:

Excellent question — this is a really good point to stop and unpack what's *actually* happening in your classification version, how it differs from the regression model, and *why* you might do this conversion at all. Let's go step-by-step in detail.

♦ 1. What the Model is Doing Conceptually

This **LSTM classification model** takes the **past 12 weeks of input features** (e.g., toxin levels, environmental forcings, etc.) and predicts **whether DSP toxin levels will exceed the regulatory limit (160 µg/kg)** for each of the next 4 weeks.

So instead of predicting **continuous toxin concentrations** like the regression model did, it's predicting **binary labels**:

Week Ahead	True Value	Model Output	Interpretation
t+1	1	0.82	High risk (above limit)
t+2	0	0.14	Low risk (below limit)
t+3	0	0.27	Low risk
t+4	1	0.76	High risk

The **LSTM layers** still learn **temporal dependencies** (patterns over time) across the 12-week input window, but the **Dense layer with a sigmoid activation** converts these learned features into *probabilities* (values between 0 and 1).

♦ 2. Step-by-Step Breakdown of the Architecture

Let's look at this part:

```
model = Sequential([
    LSTM(64, input_shape=input_shape, return_sequences=False),
```

```

Dropout(0.3),
Dense(32, activation='relu'),
Dense(horizon, activation='sigmoid') # 4 binary outputs
(0-1)
])

```

What happens in each layer:

- **LSTM(64)**
 - Reads your 12-timestep input sequences (each timestep has `num_features` variables).
 - Learns patterns like “toxin spikes tend to follow 3 weeks of rising temperature + low flow.”
 - Outputs a learned summary (64-dimensional vector).
- **Dropout(0.3)**
 - Randomly sets 30% of neuron outputs to 0 during training → reduces overfitting.
- **Dense(32, relu)**
 - Learns intermediate nonlinear patterns from LSTM’s summary.
- **Dense(HORIZON, sigmoid)**
 - Outputs 4 probabilities (one for each forecast horizon, $t+1$ to $t+4$).
 - Each output neuron says:

“Given the last 12 weeks, how likely is it that toxin exceeds 160 $\mu\text{g/kg}$ at week $t+h$?”

♦ **3. What Training Means Here**

In training:

- You provide inputs `X_train` (past 12-week windows)
- And targets `y_train` — these are **binary labels** (0 = below limit, 1 = exceedance) for the next 4 weeks.

The model minimizes **binary crossentropy loss**:

```
loss = - [ y_true * log(y_pred) + (1 - y_true) * log(1 - y_pred) ]
```

This penalizes the model for:

- Predicting high probability when toxin was actually below limit (false positive)
- Predicting low probability when toxin was above limit (false negative)

Over epochs, the optimizer (Adam) adjusts weights to **increase the probability** assigned to the correct class (0 or 1).

♦ 4. Why Convert Regression → Classification

In your regression model, the output was a continuous toxin concentration. That's great for *quantitative* forecasts — e.g., 120 µg/kg vs 300 µg/kg.

But sometimes what *really matters* is the **event** — did it cross the regulatory threshold?

Benefits of classification:

1. **Simplifies the task** — Model doesn't need to predict the *exact concentration*, just whether it's over 160 µg/kg.
2. **Better for decision-making** — Managers often care about “is it safe or not,” not “is it 190 or 210.”
3. **More robust to noise** — Concentration data can be noisy; thresholds are more stable targets.
4. **Useful when concentrations are hard to predict but direction (high vs low) is easier.**

Downsides:

1. You lose **quantitative detail** — e.g., 159 µg/kg (safe) vs 161 µg/kg (unsafe) are treated as totally different classes.
 2. The model may be less sensitive to **gradual changes** below threshold.
 3. You still depend on the **same temporal context**, so if your windowing or features are weak, both models struggle.
-

♦ 5. Why Thresholding Predictions ($y_{\text{pred}} > 0.5$)

The model outputs **probabilities** between 0 and 1.
Thresholding at 0.5 means:

- $0.5 \rightarrow$ classify as “exceedance” (1)
- $\leq 0.5 \rightarrow$ classify as “below limit” (0)

That converts **continuous probabilities** into **discrete classes** so you can compute metrics like accuracy, precision, recall, and build a confusion matrix.

If your classes are imbalanced (say only 10% of samples exceed 160 µg/kg), you might move the threshold to 0.3 or 0.7 for better recall or precision.

♦ 6. What the Plots Show

Your plot section visualizes three things per horizon:

- y_{true} : actual exceedance (0 or 1)
- y_{pred} : predicted probability
- $y_{\text{pred_bin}}$: predicted class after thresholding (0 or 1)

That gives a timeline-like view of how confident the model was, and whether it flipped correctly between above/below classifications.

♦ 7. Summary — Regression vs Classification

Aspect	Regression Model	Classification Model
Target	Continuous toxin concentration	Binary exceedance (0/1)
Loss	MSE, MAE, Huber	Binary Crossentropy
Output Activation	Linear	Sigmoid
Metric	RMSE, R^2 , MAE	Accuracy, Precision, Recall, F1, ROC-AUC
Interpretation	Predict how high toxin will be	Predict if toxin exceeds regulatory limit
Use Case	Scientific modeling, concentration forecasting	Management decisions, risk alerts

✓ In short:

The classification model is a simplified version that reframes the forecasting question from “How much toxin?” to “Will it exceed the safety limit?”

It trades precision for clarity and robustness, which can be more useful in operational or policy contexts.

If you'd like, I can walk you through a hybrid version — a **regression-classification combo model** that predicts both the concentration *and* the exceedance probability simultaneously (many environmental forecasting papers do that). Would you like to see that?

User Prompt:

```
explain to me in detail what the optimizers and hyperparameters do """ LSTM
Classification Model for DSP Toxin Exceedance Prediction
----- Predicts whether DSP toxin levels
exceed the regulatory limit (160 µg/kg) for the next 4 weeks, based on the past 12
weeks of features. Input: X_[split].npy (shape: samples, 12, num_features) Target:
y_[split].npy (shape: samples, 4) with binary values (0/1) """ import numpy as np import
os import tensorflow as tf import matplotlib.pyplot as plt from tensorflow.keras.models
import Sequential from tensorflow.keras.layers import LSTM, Dense, Dropout from
tensorflow.keras.optimizers import Adam from sklearn.metrics import confusion_matrix,
classification_report #
```

```

===== # ---
Configuration --- #
===== VARIANT =
"univariate" # or 'univariate', 'bivariate' DATA_DIR =
f"../preprocessed_classification/{VARIANT}" OUTPUT_DIR =
f"results/{VARIANT}_classification" os.makedirs(OUTPUT_DIR, exist_ok=True)
TIME_STEPS = 12 HORIZON = 4 EPOCHS = 100 BATCH_SIZE = 32 LR = 0.001
REG_LIMIT = 160 # µg/kg threshold (for reference only) #
===== # --- Load
Datasets --- #
===== print(f"\n
Loading preprocessed data from {DATA_DIR}...") X_train =
np.load(os.path.join(DATA_DIR, "X_train.npy")) y_train =
np.load(os.path.join(DATA_DIR, "y_train.npy")) X_val = np.load(os.path.join(DATA_DIR,
"X_val.npy")) y_val = np.load(os.path.join(DATA_DIR, "y_val.npy")) X_test =
np.load(os.path.join(DATA_DIR, "X_test.npy")) y_test = np.load(os.path.join(DATA_DIR,
"y_test.npy")) print(f"Shapes -> X_train: {X_train.shape}, y_train: {y_train.shape}")
num_features = X_train.shape[2] #
===== # --- Build
LSTM Classification Model --- #
===== def
build_lstm_classifier(input_shape, horizon): model = Sequential([ LSTM(64,
input_shape=input_shape, return_sequences=False), Dropout(0.3), Dense(32,
activation='relu'), Dense(horizon, activation='sigmoid') # 4 binary outputs (0–1) ])
model.compile( optimizer=Adam(learning_rate=LR), loss='binary_crossentropy',
metrics=['accuracy'] ) return model model = build_lstm_classifier((TIME_STEPS,
num_features), HORIZON) model.summary() #
===== # --- Train
Model --- # =====
print("\n Training model...") history = model.fit( X_train, y_train, validation_data=(X_val,
y_val), epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1 ) #
===== # ---
Evaluate --- #
===== print("\n 
Evaluating on test set...") y_pred = model.predict(X_test) y_pred_bin = (y_pred >
0.5).astype(int) # --- Per-horizon metrics --- for h in range(HORIZON): print(f"\n===
Horizon t+{h+1} ===") cm = confusion_matrix(y_test[:, h], y_pred_bin[:, h])
print("Confusion Matrix:") print(cm) print("\nClassification Report:")
print(classification_report(y_test[:, h], y_pred_bin[:, h], digits=3)) #
===== # --- Save
Model and Results --- #
=====
model.save(os.path.join(OUTPUT_DIR, "lstm_classifier.keras"))
np.save(os.path.join(OUTPUT_DIR, "y_pred.npy"), y_pred)
np.save(os.path.join(OUTPUT_DIR, "y_pred_bin.npy"), y_pred_bin) print(f"\n  Model
and predictions saved in {OUTPUT_DIR}") print("\n  Plotting predicted vs true

```

```
values...) for h in range(HORIZON): plt.figure(figsize=(10, 4)) plt.plot(y_test[:, h],
label="True", marker='o', linestyle='--', alpha=0.7) plt.plot(y_pred[:, h], label="Predicted
Prob.", color='orange', linewidth=2) plt.plot(y_pred_bin[:, h], label="Predicted Class",
color='red', linestyle=':', alpha=0.7) plt.title(f"LSTM Classification — Horizon t+{h+1}")
plt.xlabel("Sample Index") plt.ylabel("Exceedance (0=Below, 1=Above)") plt.ylim(-0.1,
1.1) plt.legend() plt.grid(True, linestyle='--', alpha=0.4) plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, f"y_pred_vs_true_{h+1}.png")) plt.close()
```

ChatGPT said:

Excellent — this is one of the most important things to understand when training neural networks like your **LSTM classifier**, because optimizers and hyperparameters **determine how the model actually learns** from your data.

Let's go through this in **deep detail**, using your specific code as the reference point.

♦ 1. The Optimizer — What It Does

When you compile your model:

```
model.compile(
    optimizer=Adam(learning_rate=LR),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

you're telling Keras **how to update the model's weights** after each batch of training.

An **optimizer** is essentially an algorithm that minimizes the **loss function** — i.e., it adjusts the parameters (weights, biases) of the LSTM and Dense layers so that your predictions get closer to the true labels.

Think of it like this:

After every prediction, the optimizer looks at how wrong the model was (the loss), and slightly “nudges” the internal weights in the right direction.

♦ 2. The Specific Optimizer — Adam

You're using **Adam (Adaptive Moment Estimation)**, one of the most widely used optimizers for deep learning, especially for sequence models like LSTMs.

Adam combines two key ideas:

- **Momentum:** remembers the direction of past weight updates (to keep moving in that direction).
- **Adaptive learning rate:** automatically scales how big each step should be for each parameter.

In simpler terms:

Adam takes bigger steps in directions that seem consistently good, and smaller steps where things are uncertain or noisy.

Adam's main hyperparameters:

Parameter	Default	What It Does	Analogy
<code>learning_rate</code>	0.001	Controls how large each update step is	"How fast you walk toward the goal"
<code>beta_1</code>	0.9	Controls momentum (how much of the past gradients to remember)	"How much you trust your past direction"
<code>beta_2</code>	0.999	Controls the moving average for adaptive scaling	"How much you smooth short-term noise"
<code>epsilon</code>	1e-7	Prevents division by zero	A safety cushion

You only set:

`LR = 0.001`


```
optimizer = Adam(learning_rate=LR)
```

which is often a good default. But tuning the **learning rate** can drastically change model performance.

♦ 3. The Loss Function

```
loss='binary_crossentropy'
```

The **loss function** measures how far the model's predictions are from the true labels.

For binary classification (predicting 0 or 1), **binary crossentropy** is standard:

$$L = -[y \log(p) + (1-y) \log(1-p)]$$

where

- y = true label (0 or 1)
- p = predicted probability (output of sigmoid)

So:

- If the model predicts $p=0.9$ for a true $y=1$, loss is small.
- If it predicts $p=0.1$ for a true $y=1$, loss is huge.

The optimizer's job is to **minimize this loss** across all samples.

♦ 4. Hyperparameters — The Core Training Controls

These are the knobs you can adjust that directly affect how your model learns and generalizes.

① Learning Rate (LR)

```
LR = 0.001
```

- Determines how much the optimizer adjusts weights each time.
- **Too high:** model overshoots the best solution (unstable loss, doesn't converge).
- **Too low:** model trains very slowly or gets stuck in suboptimal points.

✓ Common values to try: $1e-2$, $1e-3$, $1e-4$

② Epochs

EPOCHS = 100

- One epoch = one full pass through the training dataset.
- More epochs → model sees data more times → can learn deeper patterns.
- But after a point, it may start overfitting (memorizing the data).

✓ You can watch the **validation loss curve** to know when to stop (early stopping is often used).

③ Batch Size

BATCH_SIZE = 32

- The number of samples processed before the optimizer updates weights once.
- Smaller batches → more frequent updates → noisier but can generalize better.
- Larger batches → smoother gradient → more stable but may overfit.

✓ Typical choices: 16, 32, 64, 128
LSTMs usually perform best with smaller batches due to sequential dependencies.

④ LSTM Units

`LSTM(64, ...)`

- This defines how many memory cells the LSTM has — essentially the *capacity* to remember temporal patterns.
- More units → model can capture more complex dependencies, but needs more data to train properly.
- Fewer units → faster training but may underfit.

✓ Common test values: 32, 64, 128

⑤ Dropout Rate

`Dropout(0.3)`

- Randomly “drops out” 30% of neurons during training.
- Prevents the model from depending too much on specific neurons → reduces overfitting.

✓ Common values: 0.2–0.5

⑥ Dense Layer Size

`Dense(32, activation='relu')`

- Defines how many neurons to use after LSTM.
- Think of it as the “compression” layer that distills what the LSTM learned before final decision-making.

✓ Common test range: 16–128 neurons.

⑦ Activation Functions

```
LSTM(..., activation='tanh')  
Dense(32, activation='relu')  
Dense(1, activation='sigmoid')
```

Each activation controls how the signal flows through the network:

- **tanh**: squashes between -1 and 1 → good for sequential data.
- **relu**: keeps positive values, zeroes out negatives → efficient and fast.
- **sigmoid**: maps output to [0,1] → interpretable as probability.

Changing these affects how non-linear relationships are modeled.

⑧ Metric

```
metrics=['accuracy']
```

- Used only for monitoring, **not for training**.
- In binary classification, accuracy = fraction of correct predictions.

You can add more metrics like:

```
metrics=['accuracy', tf.keras.metrics.Precision(),  
tf.keras.metrics.Recall()]
```

- to get more detailed insight (especially if classes are imbalanced).
-

♦ 5. How All These Work Together

Each epoch, for every batch:

1. Model predicts probabilities via forward pass.
2. `binary_crossentropy` computes the loss.
3. **Adam optimizer** calculates gradients — how much each weight contributed to the error.
4. Updates weights using these gradients scaled by `learning_rate`.
5. The process repeats until all batches are processed → one epoch complete.

Then:

- The model tests its performance on the **validation** set.
- If validation loss stops improving → you may need to tune hyperparameters (e.g., lower LR or add dropout).

♦ 6. Practical Hyperparameter Tuning Strategy

Here's a typical order to tune:

Stage	Parameter	Reason
①	Learning Rate (0.001 → 0.0001 → 0.01)	Biggest effect on training dynamics
②	LSTM units (32, 64, 128)	Adjust model capacity
③	Batch size (16, 32, 64)	Impacts gradient stability

4	Dropout rate (0.2–0.5)	Regularization
5	Epochs	Stop when validation loss plateaus
6	Dense layer sizes and activations	Fine-tuning layer capacity

♦ 7. Summary Table

Type	Parameter	Controls	Key Effect
Optimizer	Adam	How weights update	Faster, smoother convergence
Learning rate	LR	Step size per update	Stability vs speed
Loss	binary_crossentropy	What to minimize	Classification accuracy
Batch size	BATCH_SIZE	Samples per update	Training stability
Epochs	EPOCHS	Number of passes	Convergence depth
LSTM units	64	Temporal capacity	Pattern complexity

Dropout	0.3	Regularization	Prevent overfitting
Dense units	32	Feature abstraction	Decision sharpness
Activations	tanh / relu / sigmoid	Nonlinearity	Learn complex relations

If you'd like, I can help you set up a **grid search or Bayesian optimization** (using Keras Tuner) to automatically test and compare different hyperparameter combinations to find the best-performing model — would you like me to show that next?

User Prompt:

i think i am confusing the optimizers and activation functions, also explain learning rate to me so that i can explain it in detail

ChatGPT said:

Perfect — this is actually one of the most common confusions when people start doing deep learning.

Let's go step by step and **clearly separate optimizers, activation functions, and learning rate**, then go deep into **how learning rate works** (with an intuitive and mathematical explanation).

1. The Key Distinction

Think of your neural network as a **machine learning factory**:

- **Activation functions** control **how signals flow *inside*** the network — they decide how neurons transform inputs into outputs (the shape of the mapping).
- **Optimizers** control **how the network learns** — how the weights change after each example or batch to reduce the loss.
- **Learning rate** is a setting **inside the optimizer** — it determines how big those changes (updates) are.

Let's unpack those.

⚡ 2. Activation Functions — "How neurons fire"

An **activation function** defines the output of each neuron given its input. It's applied **inside the network**, between layers.

Without them, every layer would just be a linear transformation — and stacking linear layers wouldn't make the model more expressive.

Activations add **nonlinearity**, allowing the model to learn complex, nonlinear relationships.

♦ Common activation functions and their behaviors

Function	Formula	Range	Typical Use	Intuition
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	$[0, \infty)$	Hidden layers	Keeps positive signals, zeroes negatives; efficient and helps gradients flow well.
Leaky ReLU	$f(x) = x$ if $x > 0$ else ax	$(-\infty, \infty)$	Hidden layers	Like ReLU, but allows small negative slope to avoid "dead neurons."
tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$[-1, 1]$	RNNs / LSTMs	Squashes inputs between -1 and 1; good for centered, smooth outputs.
sigmoid	$1 / (1 + e^{-x})$	$[0, 1]$	Output for binary classification	Converts output to probability.

🧠 Analogy:

Activations are like “filters” — they decide what patterns to keep and what to ignore in each neuron’s output.

In your model:

```
Dense(32, activation='relu')
```

```
Dense(horizon, activation='sigmoid')
```

- The `relu` layer extracts complex nonlinear features.
- The `sigmoid` layer turns the final scores into probabilities (since we want 0–1 predictions).

🔧 3. Optimizers — “How the network learns”

Once you define how your model computes outputs and loss, you need a method to **update weights** so the loss decreases over time — that’s what the optimizer does.

The optimizer sits **outside the network** and controls **how to adjust weights** based on how wrong the model was.

♦ Common optimizers

Optimizer	Key Idea	Strengths	Weaknesses
SGD (Stochastic Gradient Descent)	Updates weights in the opposite direction of the loss gradient.	Simple, robust.	Can be slow; sensitive to learning rate.
Momentum SGD	Adds momentum (remembers past updates).	Faster convergence, smooths oscillations.	Still sensitive to LR.

Adam (Adaptive Moment Estimation)	Uses momentum + adaptive scaling per parameter.	Fast, good defaults for most models.	Can sometimes overfit or overshoot if LR too high.
RMSProp	Like Adam but simpler; adjusts step size per parameter.	Works well in RNNs.	Needs tuning.

In your model:

```
optimizer = Adam(learning_rate=LR)
```

This means:

"Use Adam to automatically adjust how much each weight moves after each batch, with an initial step size of **LR**."

4. Learning Rate — “How big the step is”

The **learning rate (LR)** is the most critical hyperparameter for optimization.

It determines **how much to change the model’s weights** during training in response to the gradient (the loss slope).

♦ The math

When updating weights:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

Where:

- w : the weight being updated
- η : learning rate
- $\frac{\partial L}{\partial w}$: gradient of the loss with respect to the weight

♦ The intuition

Imagine your loss function as a **mountain landscape**, and your model is trying to reach the **lowest valley (minimum loss)**:

- The **gradient** tells you *which direction* is downhill.
 - The **learning rate** tells you *how big a step* to take.
-

♦ Case 1: Learning rate too high

- Steps are too large — you overshoot the valley.
- The loss may bounce around or even explode.
- Model fails to converge.

🧠 Think of it like sprinting downhill — you overshoot the target and fall.

♦ Case 2: Learning rate too low

- Steps are tiny — you slowly creep toward the valley.
- The loss decreases very slowly, or gets stuck in local minima.

🧠 Like walking with baby steps — you'll eventually get there but waste a lot of time (and may stop before reaching the best point).

♦ Case 3: Just right

- Smooth convergence to the minimum.
- Loss steadily decreases, then stabilizes.

🧠 Like walking briskly downhill — not too fast, not too slow.

♦ Visual intuition (if we could plot it)

If you plot loss vs epoch:

- **Too high LR:** loss oscillates or diverges upward.
 - **Too low LR:** loss decreases slowly and plateaus early.
 - **Good LR:** loss decreases smoothly and stabilizes near a minimum.
-

♦ Typical learning rate ranges

Optimize r	Common range
Adam	$1e-4 \rightarrow 1e-3$ (0.0001–0.001)
SGD	$1e-3 \rightarrow 1e-2$ (0.001–0.01)
RMSProp	$1e-4 \rightarrow 1e-3$
Adagrad	$1e-3$

♦ Advanced learning rate techniques

You can also *adapt* the learning rate during training:

- **Learning rate schedules:** gradually reduce LR over epochs.
- **Cyclic LR:** oscillates between bounds to escape local minima.
- **Adam's adaptive LR:** automatically scales LR per weight.

Example:

```
tf.keras.optimizers.schedules.ExponentialDecay(  
    initial_learning_rate=0.001,  
    decay_steps=10000,  
    decay_rate=0.96  
)
```

Summary Comparison

Concept	Operates On	Role	Controls	Example
Activation function	Inside neurons	Controls output shape	Nonlinearity	ReLU, tanh, sigmoid
Optimizer	On weights	Updates weights to minimize loss	Learning dynamics	Adam, SGD
Learning rate	Inside optimizer	Step size of updates	Training speed/stability	0.001

In short:

- **Activations** decide *how neurons transform information*.
- **Optimizers** decide *how the network learns from mistakes*.
- **Learning rate** decides *how fast or slow it learns*.

If you want, I can show you an **interactive intuition** (with code or plots) of how different learning rates affect convergence on the same dataset — would you like me to do that next?